前言：為什麼用多 Agent 寫一本多 Agent 書

生成式 AI 已經從問答工具，逐步變成能讀檔、改程式、執行命令、產出文件的工程協作者。可是，當任務從「回答一個問題」變成「完成一個長程專案」時，單一 Chatbot 很快會遇到限制：上下文太大、角色衝突、驗證不足、修改範圍失控，以及缺少可追蹤交付物。

本書選擇一個自我指涉的實戰題目：用多 Agent 團隊，製作一本介紹多 Agent 系統的科技書。這個題目同時包含規格制定、程式範例、正文寫作、技術審稿、排版建置、封面與發布流程，剛好能展示多 Agent 協作的完整生命週期。

本書不追求一次介紹所有 AI Agent 框架，也不把模型能力神化。相反地，本書關注工程化問題：如何分工、如何限制權限、如何驗證結果、如何記錄決策、如何在出錯時停止，以及如何把產物做成可重跑的發布流程。

作者簡介

Happy eBook Authors 是一個專注於 AI 輔助寫作、技術出版自動化與數位學習內容製作的創作團隊。團隊關注如何把 AI 工具真正落地到可交付的出版流程中：從選題、規格、範例程式碼、審稿、排版、封面到上架素材，皆以可驗證、可重跑、可維護為核心原則。

在本書中，Happy eBook Authors 以「用多 Agent 團隊製作一本多 Agent 書」作為實驗場，示範如何把 AI Agent 從單次問答工具，組織成能協作、能審查、能測試、能發布的工程化團隊。這也代表團隊的出版理念：電子書不只是靜態內容，而可以是一個包含程式碼、測試、排版與 release artifacts 的可持續產品。

本書適合誰

本書適合以下讀者：

- 已經使用過 ChatGPT、Claude、Codex、OpenCode 或其他 AI coding 工具的工程師。
- 想把 AI Agent 用於長程專案，而不只是單次問答的人。
- 想自動化科技寫作、審稿、排版與發布流程的作者。
- 想理解 Herdr、Pi、OpenCode、Hermes、Codex 與 Antigravity 如何分工協作的人。
- 想把多 Agent 概念落地成可測試範例與可建置產物的實作者。

本書不適合誰

本書不適合以下需求：

- 完全沒有命令列與 Python 經驗的初學者。
- 只想閱讀 AI 概念，而不想執行範例程式碼的人。
- 期待完整 LLM 數學理論或模型訓練教材的人。
- 期待所有流程都能無人工審查、完全自動上架的人。

如何閱讀本書

建議閱讀方式如下：

1. 先讀第 1 章，理解為什麼需要多 Agent 分工。
2. 讀第 2 章，建立監管中樞與狀態觀念。
3. 讀第 3 至第 6 章，理解 Pi、OpenCode、Hermes、Codex 各自的角色。
4. 讀第 7 章，把前面角色串成出版流水線。
5. 讀第 8 章，完成排版、發布與 release manifest。
6. 回到 `examples/` 目錄，逐章執行測試。

如果你是專案負責人，可以優先閱讀第 1、2、7、8 章；如果你是工程實作者，可以優先閱讀第 3、4、6 章；如果你負責品質管理，可以優先閱讀第 5 章與 `qa/reports/`。

如何使用本書中的程式碼

本書中的程式碼不是要求讀者逐段複製貼上。閱讀時，請先把程式碼當作架構說明的一部分，用來理解 Agent 之間如何通訊、驗證與交付。

書中程式碼大致分成三類：

1. 概念片段：出現在章節正文中，用來說明資料結構、狀態機、品質閘門或流程設計。這類程式碼以理解為主，不一定需要立即執行。
2. 完整範例專案：放在 GitHub release zip 的 `examples/` 目錄。若你想實際執行，請下載範例專案包，進入對應的 `examples/chXX_*` 目錄執行。
3. 維護者腳本：放在 `scripts/` 目錄，用於驗證全書範例、產生 PDF/EPUB、打包 release。這些命令適合進階讀者或專案維護者，不是第一次閱讀時的必要步驟。

簡單來說：一般讀者先看懂即可；想實作的讀者下載 GitHub 範例專案後跑 `examples/`；想重現出版流程的讀者再使用 `scripts/`。

最短上手路徑

如果你是第一次閱讀本書，不需要一開始就跑完整發布流程，也不需要立刻準備整個專案目錄。建議先閱讀第 1 章的程式片段，理解 `MessageEnvelope`、`MessageBus`、`PlannerAgent`、`CoderAgent` 與 `ReviewerAgent` 如何協作。

如果你想動手執行範例，請先到 GitHub 下載本書範例專案包：

- GitHub repository: `https://github.com/t945935/multi-agent-book`
- Release zip: `https://github.com/t945935/multi-agent-book/releases/tag/v1.0.0`

下載並解壓縮 release zip 後，再進入專案目錄執行最小範例：

```
cd multi-agent-book-v1.0.0/examples/ch01_multi_agent_baseline
python3 -m pytest
python3 main.py
```

這兩個命令會驗證第 1 章範例，並執行一個最小多 Agent 協同流程。若你只是透過電子書閱讀器閱讀，沒有下載範例專案包，可以先略過命令；每章會用文字說明核心設計，命令只是給想動手驗證的讀者使用。

如果你是專案維護者，或想確認整本書從範例、排版到發布打包都能重現，再回到範例專案根目錄執行完整驗證：

```
bash scripts/test_all.sh
PATH="$PWD/bin:$PATH" python3 scripts/build.py
python3 scripts/build_epub.py
python3 scripts/package_release.py --version 1.0.0
```

這組命令會分別驗證全書範例、建置 PDF、產生 EPUB，並建立 release manifest 與 ZIP 發布包。它不是讀者的第一個必要步驟，而是本書專案的端到端健康檢查。

執行環境

本書範例以 Python 3.12 與標準庫為主，盡量降低外部依賴。建議環境：

- Linux 或 macOS shell 環境。
- Python 3.12 以上。
- pytest。
- Typst，用於產生 PDF。

在本專案中，可使用下列命令驗證全書範例：

```
bash scripts/test_all.sh
```

產出 PDF：

```
PATH="$PWD/bin:$PATH" python3 scripts/build.py
```

打包發布檔：

```
python3 scripts/package_release.py
```

本書的限制

本書採取最小可執行範例，而不是完整商業平台實作。許多範例故意保持短小，目的是讓讀者看懂核心合約、狀態模型與品質閘門。若要投入生產環境，仍需補上更完整的權限控管、日誌保存、機密管理、CI/CD runner、錯誤追蹤與人工核准流程。

本書真正想傳達的不是某個單一工具，而是一種工程化思維：讓 AI Agent 在清楚角色、明確邊界、可驗證交付與可停止流程中協作。這也是後續所有章節的共同基礎。

第 1 章 多 Agent 協同新範式：從單一 Chatbot 到自主工程團隊

> 「當一個系統的複雜度超過單一認知實體的承受上限時，模組化與分工不是一種可選的優化，而是唯一的生存之道。」

1.1 單一 Agent 的黃金極限與崩塌點

在生成式 AI 與大語言模型（LLM）剛進入軟體工程領域時，主流做法通常是建立一個全知全能的「對話機器人（Chatbot）」或單一 Copilot：我們給予模型整個專案的程式碼上下文，期望它能同時讀懂架構、寫出程式碼、修復漏洞並通過所有安全檢查。

然而，當專案規模從單一腳本擴展到具備數十個模組的實際生產系統時，這種「單兵作戰」模式會迅速撞上一道不可逾越的物理之牆。

可以把它想像成把一本技術書交給同一個人從頭做到尾：他要同時當作者、工程師、審稿、排版、封面設計與發行。短篇文章或許可以，但一本技術書很快會失控。多 Agent 的核心不是讓更多 AI 同時說話，而是把角色、責任與驗收方式分清楚。

| 單一 Agent | 多 Agent 團隊 | |—|—| | 一個模型處理所有任務 | 多個角色分工合作 | | 上下文巨大，容易遺漏限制 | 每個 Agent 只取得必要上下文 | | 自我審查容易產生確認偏誤 | 獨立審查角色負責質疑與驗證 | | 出錯時難以定位責任 | 可回到特定角色、階段與交付物 | | 完成訊號常依賴自然語言 | 以測試、審稿、build 作為品質閘門 |

1.1.1 上下文視窗的真實成本與注意力衰減（Lost in the Middle）

現代 LLM 雖然宣稱具備 128K 甚至超過 1M Token 的上下文視窗，但在軟體工程實踐中，「上下文容量」並不等同於「有效注意力」。

史丹佛大學與柏克萊大學的多項研究（包括著名的 Lost in the Middle 現象）表明：當上下文長度急遽膨脹時，模型對於位於提示詞中間區域的指令與細節約束，召回準確率（Retrieval Accuracy）會呈現顯著的 U 型衰減。在軟體開發場景中，這意味著：

1. 指令覆蓋（Prompt Drift）：當你向單一 Agent 輸入了 5 萬字的系統規格、API 文件以及 20 個檔案的原始程式碼時，它會開始忽略邊界條件（例如「請保持向下相容」或「使用無狀態認證」）。

2. Token 成本與延遲二次方增長：自注意力機制（Self-Attention）的時間與計算複雜度本質上隨著 Token 數量劇增。每次即使只修改一行程式碼，都需要將數萬 Token 重新餵入模型，造成昂貴的 API 帳單與數十秒的響應等待。

```
graph LR
    A["單一 Agent (All-in-One)"] -->|"巨大 Context (100k+ Tokens)"| B["LLM 核心"]
    B -->|"注意力衰減 (Lost in the Middle)"| C["遺漏架構約束"]
    B -->|"高昂延遲與 Token 成本"| D["頻繁超時與輸出幻覺"]
```

1.1.2 角色衝突難題（The Persona Paradox）

除了計算維度的瓶頸，單一 Agent 還有一個更嚴重的認知維度缺陷——角色動機衝突。

在傳統軟體工程中，為什麼「開發人員（Developer）」與「品質保證工程師（QA/Tester）」通常由不同人擔任？因為這兩種角色具有相互對立的心智模型：

- 開發者的動機：最小阻力實作功能、證明程式碼能跑、樂觀傾向。
- QA 與審查員的動機：邊界極限測試、挑出邏輯瑕疵、悲觀質疑傾向。

如果你要求同一個 LLM：「請為這個系統編寫程式碼，並同時找出你自己程式碼中的潛在弱點」，模型會傾向於給出確認偏誤（Confirmation Bias）——它會認為自己的推導已經很完美，從而給出浮於表面的審查報告。

解決之道，就是多 Agent 分工（Multi-Agent Architecture）： 讓專門的 Agent 只承擔單一職責、持有最小化上下文，並透過明確的通訊合約互相協作與制衡。

1.2 多 Agent 協同的架構模式

將單一巨石型 Agent 拆解為團隊後，核心挑戰轉移到了「通訊與組織拓撲」。目前業界最主流的三種模式如下：

1.2.1 集中式編排（Orchestrator-Worker Pattern）

由一個核心的 Orchestrator（指揮官）Agent 接管使用者的原始需求，將其拆解為子任務（Sub-tasks），並指派給下游的專業 Worker（如程式碼編寫者、測試者）。Worker 完成後向 Orchestrator 回報，由其統整結果。

- 優點：流程高度可控、狀態集中管理。
- 缺點：Orchestrator 容易成為單點吞吐瓶頸；若 Orchestrator 規劃錯誤，整條鏈路將偏離方向。

1.2.2 流水線式傳遞（Linear Pipeline Pattern）

如同傳統 Unix 管道（Pipeline）：`Agent A | Agent B | Agent C`。前一個 Agent 的輸出直接作為下一個 Agent 的輸入。例如：架構師（生成 PRD）→ 工程師（生成程式碼）→ 審查員（審批發布）。

- 優點：邊界極度清晰、單向無分支，極易做自動化 CI/CD 整合。
- 缺點：回溯重試（Backtracking）成本較高，若中途某一步驟失敗，需要設計明確的退回機制。

1.2.3 去中心化訊息總線（Event-Driven Message Bus）

所有 Agent 掛載在一個非同步訊息總線（Message Bus）上，透過「主題訂閱（Pub/Sub）」進行通訊。Agent 根據事件主題（例如 `task.review.failed`）自主響應並推動流程。

- 優點：系統解耦最徹底，可隨意插入新的監聽代理（例如日誌審查 Agent、資安掃描 Agent）而不破壞現有程式碼。
- 缺點：需設計嚴格的事件合約與循環偵測機制，避免 Agent 間產生非預期的對話死迴圈。

1.3 核心合約設計：Agent 如何精確通訊

在多 Agent 系統中，絕對不能依賴混亂的純文字非結構化交談，必須建立標準化的訊息信封（Message Envelope）與狀態機（State Machine）。

1.3.1 結構化事件契約（Message Envelope）

每個在系統中流轉的事件包，至少應包含發送者、接收者、主題類型以及酬載：

```
@dataclass
class MessageEnvelope:
    """結構化訊息信封 (Message Envelope Contract)"""
    topic: str
    sender: str
    recipient: str
    payload: Dict[str, Any]
    message_id: str = field(default_factory=lambda: str(uuid.uuid4())[:8])
                timestamp:    str    =    field(default_factory=lambda:
datetime.now(timezone.utc).isoformat())
    reply_to: Optional[str] = None
```

1.3.2 狀態轉換生命週期

每個參與協同的 Agent，必須具備可被觀測的狀態：

```
stateDiagram-v2
    [*] --> IDLE: 初始化
    IDLE --> WORKING: 收到指派事件
    WORKING --> WAITING_INPUT: 需要人類或同儕反饋
    WAITING_INPUT --> WORKING: 收到反饋
    WORKING --> COMPLETED: 任務成功交付
    WORKING --> ERROR: 執行異常
    COMPLETED --> IDLE: 重置等待下一個任務
```

1.4 實戰項目：50 行程式碼打造第一個多 Agent 協同最小原型

現在，我們不藉助任何沉重且快速迭代破壞相容性的外部大模型框架，純粹使用 Python 標準庫（`asyncio`, `dataclasses`）來實現一個最簡可運行的多 Agent 系統。

1.4.1 核心架構解析

本實戰範例包含三位核心成員：

1. **`PlannerAgent`**（規劃者）：分析輸入並產出分解步驟。
2. **`CoderAgent`**（工程師）：訂閱 `task.plan` 主題，生成具體可執行的程式碼。
3. **`ReviewerAgent`**（審查員）：訂閱 `task.code` 主題，以獨立視角檢查並給出審核決策。

1.4.2 訊息總線實作

```python
class MessageBus:
    """異步訊息路由中樞 (Asynchronous In-Memory Message Bus)"""

    def __init__(self, max_history: int = 1000):
          self._subscribers: Dict[str, List[Callable[[MessageEnvelope],
Coroutine[Any, Any, None]]]] = {}
        self._history: List[MessageEnvelope] = []
        self._max_history = max_history
        self._lock = asyncio.Lock()

    def subscribe(self, topic: str, handler: Callable[[MessageEnvelope],
Coroutine[Any, Any, None]]) -> None:
        if topic not in self._subscribers:
```

```
            self._subscribers[topic] = []
        self._subscribers[topic].append(handler)

    async def publish(self, envelope: MessageEnvelope) -> None:
        async with self._lock:
            self._history.append(envelope)
            if len(self._history) > self._max_history:
                self._history.pop(0)

        handlers = list(self._subscribers.get(envelope.topic, []))
        if envelope.topic != "*":
            handlers.extend(self._subscribers.get("*", []))

        if handlers:
                await asyncio.gather(*(h(envelope) for h in handlers),
return_exceptions=True)
```

1.4.3 協同執行與輸出觀察

在 `main.py` 中，我們將總線與各個 Agent 串聯起來：

```
async def main() -> None:
    bus = MessageBus()
    planner = PlannerAgent("planner", bus)
    coder = CoderAgent("coder", bus)
    reviewer = ReviewerAgent("reviewer", bus, max_turns=3)

    bus.subscribe("task.plan", coder.handle_message)
    bus.subscribe("task.code", reviewer.handle_message)

    user_prompt = "請實作一個計算兩倍數值的函數，並完成審查"
    plan = await planner.step(user_prompt)
    code_res = await coder.step()
    review_res = await reviewer.step()
```

當我們在終端執行時，可以清晰觀察到各 Agent 的狀態流轉：

```
============================================================
🚀 啟動多 Agent 協同最小原型系統 (Chapter 1)
============================================================

[使用者輸入] 請實作一個計算兩倍數值的函數，並完成審查
```

```
▶ 階段 1: Planner 開始任務規劃...
  [Planner 產出步驟] ['實作驗證模組', '加上型別註記', '編寫單元測試']

▶ 階段 2: Coder 接收規劃並生成程式碼...
  [Coder 程式碼片段]
def solve(x: int) -> int:
    return x * 2

▶ 階段 3: Reviewer 執行審查...
  [Reviewer 決策] APPROVED - 型別註記完整，邏輯通過審查。

============================================================
✅ 協同流水線成功完成！
總線記錄事件數: 3
============================================================
```

1.5 生產環境實務陷阱：死迴圈與熔斷機制（Circuit Breaker）

在多 Agent 自主協同系統中，最致命的故障模式之一是「話癆陷阱（Ping-Pong Deadlock）」：

- Coder 產出程式碼 ➔ Reviewer 提出小瑕疵 ➔ Coder 修正 ➔ Reviewer 又提出新視角 ➔ 雙方進入無窮遞迴對話。

這不僅會耗盡伺服器計算資源，更會在幾分鐘內刷爆 API 額度。因此，所有生產級 Agent 都必須內建熔斷器（Circuit Breaker）：

```python
# 熔斷保護機制：若超過最大輪數強制終止或降級處理
if self.turn_count > self.max_turns:
    result = {
        "task_id": payload["task_id"],
        "decision": "APPROVED_BY_CIRCUIT_BREAKER",
        "notes": f"達到最大審查輪數 ({self.max_turns})，強制熔斷完成。"
    }
```

1.6 從一句需求到多 Agent 任務拆解

為了讓抽象架構更具體，讓我們用本書專案本身作為例子。假設使用者只提出一句需求：「用多 Agent 團隊寫一本介紹多 Agent 實作的科技書」。如果交給單一 Chatbot，它可能會直接生成一份長文章，內容看起來完整，但很難保證程式碼可執行、章節一致、排版可建置，也不容易在後續迭代中維護。

多 Agent 團隊會把這句需求拆成一連串明確任務。首先，Orchestrator 將需求轉成 PRD，決定書名、讀者、章節數、產物格式與完成標準。接著，Hermes 根據 PRD 撰寫章節規格，確保全書邏輯與術語一致。Codex 根據每章規格建立可測試範例，並以 pytest 證明範例能跑。OpenCode 依據規格與範例撰寫正文。Pi 進行標點、Markdown、Typst 與小型樣式修補。最後，Antigravity 或出版 pipeline 執行 build，產出 PDF 與 release artifacts。

這種拆解的重點不是增加複雜度，而是讓每個成果都有明確責任人與驗收方式。PRD 是否完整，可以由章節規格覆蓋率檢查；範例是否可靠，可以由測試檢查；正文是否可信，可以由 Hermes 審稿；排版是否成功，可以由 Typst build 檢查。當每一層都有可觀測結果時，使用者不必盲目信任某個巨型回答，而能逐步確認成果。

1.7 多 Agent 系統的四個基礎元件

不論使用哪一種工具，多 Agent 系統都離不開四個基礎元件：角色、權限、交付物與品質閘門。

角色 定義代理為何存在。沒有角色的代理會互相重疊，最後每個人都想做所有事。角色應該具體，例如「章節主筆」、「程式驗證者」、「技術審稿者」、「排版修補者」，而不是籠統地稱為「AI 助手」。

權限 定義代理能碰什麼。Codex 可以修改 examples，OpenCode 可以修改 src，Hermes 維護 docs 與 qa，Pi 做精準 patch，Antigravity 管編排與發布。權限不只是安全設定，也是協作禮貌。清楚權限能避免代理互相覆蓋工作。

交付物 定義代理完成後留下什麼。好的交付物不是一句「完成了」，而是檔案、測試結果、審稿報告、build log、manifest 或摘要。交付物讓下一個代理可以接手，也讓人類可以審查。

品質閘門 定義何時可以進入下一階段。測試未通過，不應開始撰寫引用該程式的教學；審稿有重大錯誤，不應進入排版；PDF build 失敗，不應發布。品質閘門讓多 Agent 團隊不會只追求速度，而能保持可信交付。

1.8 常見反模式：讓所有 Agent 都能改所有檔案

多 Agent 專案最危險的反模式，是把每個代理都設成全能管理員。短期看起來方便，因為任何代理都能立即修任何問題；長期則會造成混亂，因為沒有人知道某個檔案為什麼被改、誰應該負責、哪個版本才是權威。

另一個反模式是缺少交接摘要。代理 A 做完工作後只留下大量終端輸出，代理 B 必須從頭閱讀所有歷史才能接手。這會讓多 Agent 協作退化成昂貴的上下文搬運。正確做法是每次交接都留下簡短但結構化的摘要：已完成什麼、修改哪些檔案、驗證結果如何、還有哪些風險。

第三個反模式是用自然語言取代狀態機。人類可以理解「差不多好了」，但自動化 pipeline 不能。系統需要明確狀態，例如 `RUNNING`、`BLOCKED`、`DONE`、`FAILED`，也需要明確測試結果，例如 PASS 或 FAIL。

第四個反模式是沒有熔斷。當兩個代理互相要求修正，或某個測試反覆失敗時，系統必須知道何時停止。停止不是失敗，而是保護資源與品質的必要機制。

1.9 你現在應該能做到

讀完本章後，你應該能：

- 說明單一 Agent 在長程任務中的上下文與角色衝突限制。
- 區分 Orchestrator、Pipeline 與 Message Bus 三種協作模式。
- 說明 Message Envelope 與狀態機為何是多 Agent 通訊基礎。
- 執行第 1 章範例，觀察 Planner、Coder、Reviewer 的協作流程。

1.10 本章小結

本章我們跳脫了傳統單一 Chatbot 的單兵思維，剖析了多 Agent 系統在認知分離與上下文精簡上的必然性。我們定義了結構化事件合約（Message Envelope），並透過 Python 標準庫完成了零依賴的 Message Bus。

在接下來的第 2 章中，我們將走出單一 Python 進程的沙盒，探討在真實作業系統與命令列環境中，如何使用新一代 AI 終端多工器 Herdr 同時監管、調度多個完全獨立運行的 Agent 進程。

課後實踐與思考題

1. 實操題：檢視 `examples/ch01_multi_agent_baseline/` 專案，執行 `python3 -m pytest test_baseline.py`，確認 3 項測試全數通過。

2. 架構題：若要為該系統增加一個「資安審查員（SecurityAuditorAgent）」，你認為它應該以「流水線模式」串在 Reviewer 之後，還是透過 MessageBus「並行訂閱」`task.code` 主題？請思考兩種方案對延遲與覆蓋率的影響。

第 2 章 終端多工與監管中樞：Herdr 架構與 Agent 狀態感知

> Herdr 將終端多工器、狀態快照與監管策略結合，讓多個代理不再只是各自輸出文字，而是成為可觀測、可接管、可復原的工程團隊。

2.1 本章導讀：多 Agent 需要一個控制塔

當我們只和單一 Chatbot 對話時，介面很單純：輸入一句話，等待一段回答，判斷結果是否可用。但是多 Agent 系統不是單線對話。它同時包含寫作代理、程式代理、審稿代理、排版代理、測試代理與總編排代理。每個代理可能在不同終端、不同目錄、不同任務階段中運行，也可能同時等待使用者核准、執行測試、產生檔案或回報錯誤。

這時真正困難的問題不是「能不能啟動很多 Agent」，而是「能不能知道每個代理現在到底在做什麼」。如果缺少監管中樞，團隊很快會陷入幾種混亂狀態：某個代理已經卡住但沒有人知道；某個代理以為任務完成，但其實測試尚未通過；某個代理在錯誤目錄修改檔案；另一個代理重複執行昂貴或危險的指令。這些問題都不是模型能力不足，而是協作系統缺少可觀測性。

Herdr 的價值可以理解為多 Agent 團隊的控制塔。它不只是把多個終端放在一起，而是讓使用者與上層編排器能夠看見每個 pane 的狀態、最後輸出、阻塞原因與是否需要人工介入。換句話說，Herdr 把終端輸出從「文字流」提升為「可治理的任務狀態」。

本章會從監管需求開始，介紹狀態模型、pane 管理、告警策略與簡化版實作，並以 `examples/ch02_herdr_orchestration/` 的程式碼展示如何把終端輸出轉換為可測試的狀態快照。需要先說清楚的是：本章範例不是 Herdr 本體實作，而是抽取 Herdr 型工具最核心的監管概念，做成一個可測試的 Python 最小模型。

```
flowchart LR
    A[使用者需求] --> B[多個 Agent 任務]
    B --> C[終端 Pane 輸出]
    C --> D[Herdr 狀態快照]
    D --> E[監管 Dashboard]
    E --> F[人工接管或自動推進]
```

2.2 架構問題：從可用到可治理

多 Agent 系統最容易被低估的問題，不是模型能不能回答，而是回答之後能不能被治理。工程團隊需要知道誰正在工作、誰被卡住、哪一個輸出可以信任，以及何時必須停止自動化並交給人類。

在一般終端中，我們看到的是連續輸出的文字。文字可以提供線索，卻不一定能直接支撐決策。例如「done」可能代表某個小步驟完成，也可能代表整個任務完成；「error」可能是可忽略的警告，也可能是阻斷流程的失敗；「waiting for input」則代表自動化流程已經無法自行前進。若系統只把這些內容當成日誌保存，上層流程就很難判斷下一步應該繼續、重試、暫停還是交給人。

因此，本章聚焦的核心，是把代理的行為從聊天紀錄提升為工程物件。工程物件必須有狀態、有輸入輸出、有驗收條件，也要能在失敗時留下足夠線索。這種設計看似繁瑣，卻是長程任務能穩定運行的必要成本。

在本書的科技書生產流水線中，Herdr 型監管中樞至少要回答下列問題：

- 哪些代理正在工作？
- 哪些代理已經完成？
- 哪些代理正在等待使用者核准？
- 哪些代理出現錯誤，需要人工檢查？
- 哪些任務可以安全交給下一個代理？
- 哪些任務雖然有輸出，但尚未通過測試或審稿？

這些問題若交給人類逐一切換終端檢查，成本會隨代理數量快速上升。兩個代理時還能靠記憶管理；五個代理時就會開始漏看訊息；十個代理以上，沒有狀態摘要幾乎無法維持穩定協作。

2.3 Herdr 與傳統 tmux 的差異

傳統 `tmux` 或 `screen` 解決的是終端多工問題：你可以建立多個 pane、切換工作區、保留長時間執行的程序。這對人類工程師很有用，但它本身不理解「代理任務」的語意。它知道某個 pane 有輸出，卻不知道那代表完成、等待、失敗還是偏離目標。

Herdr 類工具要解決的是更高一層的問題：在多個 AI coding agent 同時運作時，如何讓終端變成可監管的代理儀表板。它關心的不是只有 pane 存不存在，而是 pane 背後的代理是誰、負責什麼任務、目前狀態為何、是否應該提醒使用者，以及是否可以把工作交給下一個階段。

可以用三個層次理解差異：

1. 終端層：管理 pane、session、scrollback 與程序生命週期。
2. 狀態層：從輸出中抽取 `RUNNING`、`BLOCKED`、`DONE` 等狀態。

3. 決策層：根據狀態決定是否提醒、重試、停止、接管或推進流程。

tmux 主要位於終端層；Herdr 型監管中樞則把狀態層與決策層補上。對多 Agent 團隊來說，這個差異非常關鍵，因為大型任務的失敗通常不是單一指令失敗，而是某個交接點沒有被正確監看。

2.4 狀態模型：讓輸出變成可判斷訊號

本章範例使用四個基本狀態：

- `IDLE`：pane 沒有有效輸出，或目前沒有任務。
- `RUNNING`：代理正在輸出內容，任務看起來仍在進行。
- `BLOCKED`：代理明確表示等待輸入、等待核准或被阻塞。
- `DONE`：代理輸出完成訊號，例如 `done` 或測試 `passed`。

真實系統通常會有更多狀態，例如 `FAILED`、`REVIEW_REQUIRED`、`APPROVAL_REQUIRED`、`TIMEOUT`、`CANCELLED`。本章故意先採用簡化模型，原因是狀態模型最重要的不是一次設計得多完整，而是每個狀態都要能被穩定辨識、測試與使用。

狀態模型應避免兩種極端。第一種極端是完全沒有狀態，只依賴人類讀終端輸出。這會讓監管成本過高。第二種極端是過度細分狀態，導致每個狀態的定義都很模糊，代理與使用者都不知道該如何處理。好的狀態模型應該少而明確，並能直接對應處置方式。

例如：

- `RUNNING`：繼續觀察，不要過早打斷。
- `BLOCKED`：提醒使用者或總編排器介入。
- `DONE`：進入驗收流程，而不是直接信任結果。
- `IDLE`：確認是否尚未開始、已結束但未回報，或程序異常停止。

這裡要特別注意：`DONE` 不等於「品質合格」。它只代表該代理自稱完成或輸出中出現完成訊號。真正的合格必須由後續測試、審稿或 build 結果判斷。這也是本書反覆強調品質閘門的原因。

2.5 監管者視角：什麼時候該介入

多 Agent 系統的監管不是每看到輸出就插手。過度介入會讓代理失去連續工作能力，也會讓人類重新成為瓶頸。好的監管策略應該把介入時機限制在少數高價值情境。

第一種情境是明確阻塞。當代理輸出 `waiting for input`、`blocked`、`need approval` 之類訊息時，代表它已經無法自行推進。這時監管者應該查看任務上下文，決定給予授權、補充資訊，或要求代理停止。

第二種情境是錯誤訊號。當輸出包含 `error`、`failed`、`traceback`、`permission denied` 等字樣時，不一定代表整個任務失敗，但必須提高警覺。監管中樞可以先把 pane 標記為 `needs_attention`，再交由人類或審查代理判斷。

第三種情境是長時間無進展。代理可能沒有報錯，卻在同一段推理或同一個命令上停留太久。這種問題需要時間資訊、最後輸出時間與心跳機制。本章範例尚未實作時間維度，但在生產環境中，這是監管中樞非常重要的一部分。

第四種情境是越權或越界。若某個代理開始修改不屬於它的目錄、執行刪除命令、安裝不必要依賴，或反覆重寫大範圍檔案，監管者應該立即介入。這類問題通常無法只靠文字狀態判斷，還需要工具層權限、檔案 diff 與命令審批共同配合。

2.6 實戰範例：簡化版 HerdrSupervisor

本章範例位於：

```
examples/ch02_herdr_orchestration/
```

核心程式碼如下。這段程式不是展示用偽碼，而是測試套件實際執行的來源檔。它的目標是示範如何把 pane 的文字輸出轉換成結構化快照。

```
from __future__ import annotations
from dataclasses import dataclass, field
from enum import Enum
from typing import Dict, List

class PaneState(str, Enum):
    IDLE = "IDLE"
    RUNNING = "RUNNING"
    BLOCKED = "BLOCKED"
    DONE = "DONE"

@dataclass
class PaneSnapshot:
    pane_id: str
    agent: str
    last_line: str
    state: PaneState
    alerts: List[str] = field(default_factory=list)

class HerdrSupervisor:
    def __init__(self) -> None:
        self.panes: Dict[str, PaneSnapshot] = {}

       def ingest(self, pane_id: str, agent: str, output: str) ->
PaneSnapshot:
        text = output.strip()
```

```
        lowered = text.lower()
        if "waiting for input" in lowered or "blocked" in lowered:
            state = PaneState.BLOCKED
        elif "done" in lowered or "passed" in lowered:
            state = PaneState.DONE
        elif text:
            state = PaneState.RUNNING
        else:
            state = PaneState.IDLE
        alerts = []
        if "error" in lowered or "failed" in lowered:
            alerts.append("needs_attention")
        snap = PaneSnapshot(pane_id, agent, text.splitlines()[-1] if text
else "", state, alerts)
        self.panes[pane_id] = snap
        return snap

    def dashboard(self) -> Dict[str, int]:
        counts = {state.value: 0 for state in PaneState}
        for snap in self.panes.values():
            counts[snap.state.value] += 1
        return counts
```

這個實作有幾個值得觀察的重點。

首先，`PaneSnapshot` 是一個明確的資料合約。它把原本散落在終端中的資訊整理成 `pane_id`、`agent`、`last_line`、`state` 與 `alerts`。這讓上層編排器不需要重新解析整段輸出，只要讀取快照即可。

其次，`ingest` 是狀態抽取的入口。它接收 pane 輸出，透過簡單關鍵字判斷狀態。真實 Herdr 可以使用更複雜的規則、正規表示式、事件協定，甚至讓代理主動輸出 JSON 狀態；但在教學階段，用清楚可測的關鍵字規則更容易理解。

第三，`alerts` 與 `state` 被分開處理。這是重要設計。某個 pane 可能仍是 `RUNNING`，但輸出中已經出現 `error`；也可能是 `DONE`，但過程中有警告需要人工檢查。狀態代表流程位置，告警代表風險訊號，兩者不應混在同一欄位。

第四，`dashboard` 回傳的是聚合統計。對監管者來說，第一眼不一定需要讀每個 pane 的細節，而是想知道目前有幾個任務完成、幾個阻塞、幾個仍在運行。這就是 dashboard 的核心價值：先提供總覽，再讓使用者下鑽到問題 pane。

例如，一個四 pane 的監管畫面可以先摘要成：

```
{
  "IDLE": 0,
```

```
  "RUNNING": 2,
  "BLOCKED": 1,
  "DONE": 1
}
```

這個輸出不需要讀者知道每個 pane 的完整終端內容，但已經足以提示監管者：目前最該處理的是那個 BLOCKED 任務。

2.7 多 pane 任務分派情境

假設我們正在製作本書的第 6 章。總編排器可以把工作拆成四個 pane：

- pane-1：OpenCode 撰寫章節初稿。
- pane-2：Codex 實作 examples/ch06_sandbox_runner/。
- pane-3：Hermes 檢查章節邏輯與術語一致性。
- pane-4：Pi 修正 Markdown、Typst 與標點格式。

在沒有監管中樞時，使用者必須手動切換四個 pane，閱讀每個代理輸出，再推測整體進度。有了 HerdrSupervisor 後，每個 pane 的最後輸出可以被定期送入 ingest，轉成狀態快照。dashboard 可能顯示：一個 DONE、兩個 RUNNING、一個 BLOCKED。這時使用者就知道應該優先查看阻塞 pane，而不是平均巡視所有終端。

更進一步，監管中樞可以成為自動化 pipeline 的條件來源。例如，只有當 Codex pane 顯示 DONE，且測試結果包含 passed，OpenCode 的正文草稿才允許引用該範例。只有當 Hermes 審查沒有重大風險，Pi 才進入最後格式精修。這種規則把「看起來差不多完成」改造成「符合條件才推進」。

2.8 驗證策略

範例測試聚焦於三件事：正常路徑必須成立、錯誤或拒絕路徑必須可預期、摘要輸出必須足以讓上層編排器做決策。這比單純檢查函式回傳值更接近生產環境，因為多 Agent 系統的失敗通常發生在交接邊界。

執行方式：

```
cd examples/ch02_herdr_orchestration
PYTHONPATH=. python3 -m pytest -v
```

本章測試應該確認幾種行為。第一，空輸出會被視為 IDLE，一般文字輸出會被視為 RUNNING。第二，包含 waiting for input 或 blocked 的輸出會被視為 BLOCKED。第三，包含 done 或 passed 的輸出會被視為 DONE。第四，包含 error 或 failed 的輸出會附加 needs_attention 告警。第五，dashboard 能正確統計各狀態數量。

這些測試看似簡單，卻能守住監管中樞的基本可信度。狀態抽取如果不穩，後面的自動編排就會建立在錯誤訊號上。多 Agent 系統中最危險的失敗，不是系統明確報錯，而是錯把失敗當成功、錯把等待當完成、錯把警告當普通輸出。

2.9 生產環境陷阱與反模式

狀態漂移 是第一個常見陷阱。代理輸出看似正常，但內部任務已偏離原目標。例如原本要求修改第 2 章，代理卻開始重構整個 build script。解法是把任務合約寫入結構化資料，並讓監管中樞檢查代理是否仍在授權範圍內。

重試風暴 是第二個陷阱。上游未區分暫時失敗與永久失敗，導致反覆執行同一危險操作。例如 build 失敗是因為 Typst 語法錯誤，系統卻一直重新 build，而不是停下來修正來源檔。重試應有次數限制，也應依錯誤類型選擇不同處置。

人工接管缺口 是第三個陷阱。系統只有自動成功與自動失敗，沒有提供人類介入的中間狀態。成熟的監管中樞應該允許「暫停」、「要求確認」、「交給使用者決策」、「記錄待辦」等中間流程。

完成訊號過度信任 是第四個陷阱。很多 Agent 會在回答末尾寫「已完成」，但這不代表檔案真的正確、測試真的通過、PDF 真的能編譯。監管中樞應把完成訊號視為進入驗收，而不是驗收本身。

權限邊界模糊 是第五個陷阱。當所有代理都能讀寫所有檔案，短期看似方便，長期會造成難以追蹤的修改衝突。本書採用目錄責任制：Codex 管 `examples/`，OpenCode 管 `src/`，Hermes 管 `docs/` 與 `qa/`，Pi 做精準修補與排版，Antigravity 管編排與發布。Herdr 的監管視角應該能看見這些權限邊界是否被遵守。

2.10 實戰練習

請讀者嘗試完成以下練習：

1. 在 `PaneState` 中新增 `FAILED` 狀態，讓包含 `traceback` 或 `permission denied` 的輸出直接進入失敗狀態。
2. 為 `PaneSnapshot` 增加 `updated_at` 欄位，用來記錄最後更新時間。
3. 在 `dashboard` 之外新增 `needs_attention()` 方法，回傳所有含有告警的 pane。
4. 設計一個人工接管規則：當 `BLOCKED` pane 數量大於零時，停止後續自動部署。
5. 思考哪些狀態應由代理主動回報，哪些狀態應由監管中樞從輸出中推斷。

完成練習後，請重新執行本章測試，確認新增行為沒有破壞原有狀態判斷。若要進一步挑戰，可以把輸出格式改成 JSON line，讓代理以 `{"state":"BLOCKED","reason":"waiting approval"}` 這類結構化訊息回報進度。

2.11 你現在應該能做到

讀完本章後，你應該能：

- 定義 IDLE、RUNNING、BLOCKED、DONE 等基本 Agent 狀態。
- 判斷何時需要人工接管，何時應讓代理繼續工作。
- 使用簡化版 HerdrSupervisor 將 pane 輸出轉為 dashboard 統計。
- 說明為什麼 DONE 不等於品質合格。

2.12 本章小結

終端多工與監管中樞的重點，是把代理能力收斂成可監管的工程流程。Herdr 型工具不是單純讓你開更多終端，而是讓每個終端背後的代理變得可觀測、可判斷、可接管。

本章用簡化版 HerdrSupervisor 展示了三個核心觀念。第一，pane 輸出必須被轉換為狀態快照。第二，狀態與告警應分開建模，避免把流程位置與風險訊號混為一談。第三，dashboard 應提供整體進度總覽，讓人類監管者能把注意力放在真正需要介入的地方。

下一章會轉向 Pi 的極簡哲學與微修實戰。當 Herdr 讓我們看見多 Agent 團隊的狀態後，Pi 則示範另一種重要能力：在不破壞既有成果的前提下，以精準、小步、可審查的方式修補內容與程式。這種微修能力，是大型協作專案能穩定累積品質的關鍵。

第 3 章 極簡純粹的工程代理：Pi 的極簡哲學與微修實戰

> Pi 的價值不在於龐大框架，而在於小步、精準、可審查的檔案修改。本章建立一個微修代理，示範如何將變更限制在最小可驗證範圍內。

3.1 本章導讀：為什麼微修能力很重要

在多 Agent 團隊裡，最吸引人的畫面通常是大型自動化：代理可以一次規劃、寫程式、跑測試、產出 PDF，甚至準備上架素材。然而，真正讓大型協作能長期穩定運作的，往往不是一次大改，而是一次又一次小而準確的修補。

Pi 的定位正是這種「微修工程代理」。它不追求接管整個專案，也不預設自己要重寫大量檔案。它的核心價值是：讀懂目前狀態，找出最小修改點，用可審查、可回滾、可驗證的方式完成變更。這種哲學對書籍專案特別重要，因為書籍同時包含文字、程式碼、排版樣式與發行腳本。若每次修正錯字都重寫整章，或每次調整字型都改動整個模板，專案很快會失去可信度。

本章會以 `examples/ch03_pi_agent_extension/` 的微修工具為例，說明 exact text replacement、單一匹配、防止模糊修改、標點正規化與最小 diff 的實作思路。讀者將學會一個重要觀念：在多 Agent 系統中，能力越強的代理越需要被限制在清楚邊界內；能改很多不是美德，能只改必要部分才是工程品質。

```
flowchart LR
    A[讀取目前內容] --> B[定位唯一修改點]
    B --> C[套用最小 patch]
    C --> D[重新驗證]
    D --> E[輸出可審查摘要]
```

3.2 Pi 在多 Agent 團隊中的定位

如果把本書的多 Agent 團隊比喻成出版工作室，OpenCode 像章節主筆，Codex 像工程驗證者，Hermes 像技術主編，Antigravity 像總編排中心，而 Pi 則像一位極度謹慎的校對

與排版工程師。它不一定負責提出最大膽的架構，但負責把已經存在的成果修到更準、更穩、更一致。

Pi 適合處理的任務包括：

- 修正 Markdown 標題層級與標點。
- 調整 Typst 模板的小型樣式問題。
- 修改單一函式或測試中的明確錯誤。
- 統一術語，例如把「代理」與「Agent」的用法整理一致。
- 消除 build warning，例如替換不存在的字型名稱。
- 在不改變章節意圖的情況下微調語句。

Pi 不適合在沒有明確指示時大規模重構。若任務需要重新設計章節架構、改寫整個程式模組、調整出版 pipeline，應先交給 Plan 型代理提出方案，再由使用者或編排器核准。Pi 的優勢不是「想得最多」，而是「動得最準」。

3.3 小步修改的工程價值

在單人專案中，大範圍修改有時只是風格問題；在多 Agent 專案中，大範圍修改會變成協作風險。因為其他代理可能正在讀同一個檔案、基於舊內容撰寫審稿報告，或等待某個範例測試結果。如果 Pi 在沒有必要的情況下重寫整章，其他代理的上下文就會失效。

小步修改有三個直接好處。

第一，容易審查。當 diff 只包含必要幾行，人類可以快速判斷修改是否合理。若 diff 包含大量無關格式變動，就很難看出真正風險在哪裡。

第二，容易定位錯誤。若 build 在修改後失敗，而本次只改了一行字型設定，就能快速回到該行檢查。若一次重寫十個檔案，失敗原因就會被淹沒在大量變更中。

第三，容易與其他 Agent 協作。小 patch 比大 patch 更不容易覆蓋他人成果，也更適合在 Herdr 監管中樞中追蹤狀態。這就是為什麼 Pi 會偏好 exact text replacement，而不是模糊地說「幫我把這段改好」。

3.4 實戰範例：微修工具 micro_patch

本章範例位於：

```
examples/ch03_pi_agent_extension/
```

核心程式碼如下：

```
from __future__ import annotations
from dataclasses import dataclass
```

```
@dataclass(frozen=True)
class PatchResult:
    changed: bool
    content: str
    occurrences: int

def replace_once(content: str, old: str, new: str) -> PatchResult:
    occurrences = content.count(old)
    if occurrences != 1:
        return PatchResult(False, content, occurrences)
    return PatchResult(True, content.replace(old, new, 1), occurrences)

def normalize_fullwidth_punctuation(text: str) -> str:
    table = str.maketrans({",": ", ", ":": ": "})
    return text.translate(table)
```

這段程式的重點不在複雜，而在安全。`replace_once` 先計算 `old` 在內容中出現幾次。只有剛好出現一次時，才允許替換。若出現零次，代表上下文已改變，不能假裝成功。若出現多次，代表修改目標不唯一，也不能冒險替換。

`PatchResult` 則把結果顯式化：是否修改、修改後內容、匹配次數。這比單純回傳字串更可靠，因為上層代理可以根據 `changed` 與 `occurrences` 做決策。例如匹配零次時重新讀檔，匹配多次時要求使用者提供更精準上下文。

`normalize_fullwidth_punctuation` 示範另一種 Pi 常見任務：格式正規化。中文科技書常混用半形逗號與冒號，讀起來不夠一致。這類修改不需要大型模型重新創作，只需要穩定規則與可測試函式。

3.5 exact text replacement 的限制

精準替換並不是萬能工具。它適合修改已知片段，卻不適合處理語意模糊或結構大幅變動的任務。若使用者說「把這章寫得更好」，Pi 不應直接用 exact replacement 猜測要改哪裡；它應該先要求具體目標，或把任務交給章節主筆代理。

它也容易受到上下文變動影響。當其他代理已經修改同一段文字，原本的 `oldText` 可能不再存在。這不是壞事，而是安全機制正在發揮作用。修改失敗比錯誤修改更可取，因為失敗至少會讓監管者知道需要重新檢查。

另一個限制是重複片段。許多 Markdown 檔案會重複出現相同句子，例如章節模板中的「本章範例位於」。若只用這句作為替換目標，就可能匹配多處。正確做法是加入足夠上下文，但又不要把整個檔案都當成替換範圍。Pi 的技巧在於選擇「小而唯一」的上下文。

3.6 常見錯誤與反模式

最常見的壞情境，是使用者只要求修正一個錯字，代理卻重新排版整章、改掉標題、重寫範例，甚至順手調整與任務無關的語氣。這種修改看起來很努力，實際上非常難審查，也容易覆蓋其他 Agent 的成果。

| 壞 patch | 好 patch | |—|—| | 重寫整個檔案 | 只修改必要段落 | | 順手改風格 | 只處理要求問題 | | 找不到目標仍回報完成 | 找不到目標就拒絕 | | 多處匹配時改第一處 | 多處匹配時停止並要求更精準上下文 | | 修改後不驗證 | 根據影響範圍執行測試或 build |

第一個反模式是「順手美化」。代理本來只需要修正一個錯字，卻順手改了整段語氣、標題格式與列表順序。這會讓審查者無法判斷哪些是必要修改，哪些只是模型偏好。

第二個反模式是「全檔重寫」。全檔重寫會破壞 blame、diff 與其他代理上下文。除非使用者明確要求重構，否則 Pi 應避免覆蓋整份文件。

第三個反模式是「模糊成功」。有些工具在找不到目標時仍回報完成，或在多處匹配時替換第一處。這對多 Agent 系統非常危險，因為上層編排器會基於錯誤成功訊號繼續推進。

第四個反模式是「忽略驗證」。即使只是修改文字，也可能造成 Typst build 失敗，例如不小心引入未跳脫的語法符號。Pi 完成修改後，應根據影響範圍執行適當驗證：文字修改可重新 build，程式修改要跑測試，模板修改則至少編譯一次 PDF。

3.7 驗證策略

本章測試可用下列方式執行：

```
cd examples/ch03_pi_agent_extension
PYTHONPATH=. python3 -m pytest -v
```

測試重點包括：

- 目標字串只出現一次時，替換應成功。
- 目標字串出現多次時，替換應拒絕。
- 標點正規化應穩定產生預期結果。

這些測試代表一種協作承諾：Pi 不會在不確定時假裝確定。對多 Agent 團隊而言，這比「看起來很聰明」更重要。

3.8 實戰練習

請讀者嘗試完成以下練習：

1. 為 `replace_once` 增加 `preview` 模式，只回傳將會修改的摘要，不直接修改內容。
2. 增加 `replace_many_exact`，但要求每個 `old` 都必須唯一，否則整批拒絕。

3. 為 Markdown 標題增加檢查器，確認章節中只有一個一級標題。
4. 實作一個術語正規化函式，把「多代理」統一成「多 Agent」.
5. 修改一個 Typst 模板設定後，重新執行 `scripts/build.py` 確認 PDF 可產出。

3.9 實務案例：修正排版警告

在本書專案中，Pi 的角色可以用一個實際例子說明。建置 PDF 時，Typst 曾回報 code block 字型警告，原因是模板指定了系統中不存在的 `JetBrains Mono` 與 `Fira Code`。這不是需要重構整個模板的大問題，也不需要改寫所有章節；它只需要定位到 `templates/book.typ` 中的 code block 字型設定，改成系統可用的 `DejaVu Sans Mono`，再重新 build 確認警告消失。

這正是 Pi 的理想任務：問題具體、範圍小、修改點唯一、驗證方式明確。如果由大型寫作代理處理，它可能順手調整整個模板風格；如果由沒有上下文的工具處理，可能只替換第一個字型名稱而留下其他警告。Pi 的做法則是先讀檔、找唯一位置、做最小替換、重新執行 build。

這個案例也說明，微修不是低價值工作。一本書的品質常由大量小細節決定：字型警告、標題層級、空白行、標點一致性、路徑拼字、code fence 是否關閉。這些問題單獨看都很小，但累積起來會決定讀者是否覺得專案專業。

3.10 微修任務的交付格式

Pi 完成任務後，應用固定格式回報。建議包含四項：修改檔案、修改內容、驗證命令、驗證結果。例如：「修改 `templates/book.typ`，將 code block 字型改為 `DejaVu Sans Mono`；執行 `python3 scripts/build.py`；PDF 成功產出且無字型 warning。」

這種回報格式讓 Hermes 可以審查，讓 Antigravity 可以更新任務狀態，也讓使用者知道修改是否真的完成。若驗證失敗，Pi 不應粉飾結果，而要明確說明失敗命令與錯誤訊息。

3.11 何時不要使用 Pi

不是所有問題都適合 Pi。若需求是重新設計整本書章節、建立全新 CI/CD 架構、決定商業出版策略，Pi 不應直接執行。這些任務需要 Plan 模式、技術主編或總編排者先做決策。Pi 適合在決策已明確後，把具體修改落到檔案上。

這種克制是多 Agent 團隊成熟的標誌。每個角色知道自己能做什麼，也知道什麼時候該停下來請其他角色接手。

3.12 微修檢查清單

執行微修前，先確認修改目標是否唯一、上下文是否最新、是否只改必要範圍、是否知道驗證方式。執行後，確認 diff 是否小而可讀，並把驗證結果回報給下一個 Agent。這份清單能讓 Pi 的工作保持穩定，不會從微修滑向無邊界重寫。

3.13 你現在應該能做到

讀完本章後，你應該能：

- 說明為什麼微修比大範圍重寫更適合多 Agent 協作。
- 使用 `replace_once` 避免模糊替換。
- 判斷好 patch 與壞 patch 的差異。
- 為微修任務留下修改內容、驗證命令與驗證結果。

3.14 本章小結

Pi 的極簡哲學提醒我們：在多 Agent 系統中，精準比炫技重要，小步比大改安全，可審查比自信輸出可靠。當 Herdr 讓我們看見各代理狀態後，Pi 提供的是在既有成果上穩定累積品質的能力。

下一章將討論 OpenCode 的 Plan 與 Build 雙模式。若說 Pi 擅長已知修改點的精準修補，OpenCode 則適合把較大的寫作或工程任務先拆成計劃，再進入受控建置階段。

第 4 章 雙模式架構實戰：OpenCode 的 Plan 與 Build 拆解

> OpenCode 的核心分工是先規劃再建置。Plan 模式保護方向，Build 模式落實修改，兩者之間以任務合約交接。

4.1 本章導讀：先想清楚，再開始改

大型任務最常見的失敗，不是代理不會改檔案，而是太快開始改檔案。使用者提出「擴充這本書」、「重構這個模組」、「建立出版流水線」時，任務其實包含許多隱含決策：要改哪些檔案、哪些內容不能碰、如何驗證、誰負責審查、失敗時如何回退。如果代理跳過這些問題直接進入修改，很容易產生看似勤奮但方向錯誤的成果。

OpenCode 的 Plan / Build 雙模式，就是為了解決這個問題。Plan 模式負責理解目標、拆解步驟、列出風險與驗證方式；Build 模式只在計劃被核准後執行具體修改。這種分工把「思考」與「動手」分開，讓大型任務可以被審查、被暫停、被交接。

在多 Agent 團隊中，OpenCode 常扮演章節主筆與方案整理者。它可以根據 Hermes 的章節規格、Codex 的範例程式、Pi 的格式規則，產生正文草稿或修改方案。但它不應在沒有邊界的情況下自由改動所有檔案。Plan / Build 的價值正在於建立這條邊界。

```
flowchart LR
    A[使用者目標] --> B[Plan 模式]
    B --> C[任務合約]
    C --> D[核准]
    D --> E[Build 模式]
    E --> F[驗證與交付]
```

4.2 Plan 模式：把模糊需求變成任務合約

Plan 模式的輸出不應只是「我會處理」。好的計劃至少要回答六個問題。

第一，目標是什麼？例如「將第 3 章從短版骨架擴充為 3000 字以上的教學章節」。目標要可驗收，而不是抽象地說「寫得更好」。

第二，範圍是什麼？本次只修改 `src/ch03.md`，還是也會修改範例程式、測試、模板與 README？範圍越清楚，越能避免代理越界。

第三，步驟是什麼？例如先讀原文，再對照範例程式，接著重寫章節，最後統計字數與 build PDF。步驟可以讓使用者看見代理的工作路徑。

第四，風險是什麼？擴充 Markdown 可能破壞 Typst 轉換；修改程式可能造成測試失敗；調整模板可能讓 PDF 編譯失敗。計劃應提前列出風險。

第五，驗證方式是什麼？完成後要跑 `scripts/test_all.sh`、`scripts/build.py`，還是只需檢查單章字數？沒有驗證方式的計劃，不應進入 Build。

第六，停止條件是什麼？若遇到刪檔、重大重構、需要外部金鑰或大量網路下載，應停止並要求使用者核准。

4.3 Build 模式：只執行已核准的計劃

Build 模式的重點是紀律。它不應重新發明目標，也不應任意擴張範圍。若 Plan 說只修改第 4 章，Build 就不應順手改第 5 章。若 Plan 說不碰測試，Build 就不應重新設計測試框架。

這並不代表 Build 模式不能思考。實作過程中一定會遇到細節判斷，例如某段文字放在哪一節比較合理、某個標題要不要改名、某個範例要不要補充說明。Build 可以做局部判斷，但必須符合原計劃的意圖與範圍。若發現計劃本身有問題，正確做法是停止並回報，而不是靜默改變計劃。

在本書專案中，OpenCode 的 Build 常見流程如下：

1. 讀取章節規格與既有正文。
2. 對照 examples 中的可執行程式。
3. 補足問題背景、核心概念、實作解說、反模式與練習。
4. 保留與其他章節一致的語氣和結構。
5. 完成後交由 Hermes 審稿、Pi 修格式、Codex 驗證程式。

4.4 實戰範例：workflow.py

本章範例位於：

```
examples/ch04_opencode_workflow/
```

核心程式碼如下：

```
from __future__ import annotations
from dataclasses import dataclass
```

```
from typing import List

@dataclass(frozen=True)
class Plan:
    goal: str
    steps: List[str]
    approved: bool = False

@dataclass(frozen=True)
class BuildArtifact:
    files: List[str]
    summary: str

def create_plan(goal: str) -> Plan:
    return Plan(goal, ["inspect", "design", "modify", "verify"], False)

def approve(plan: Plan) -> Plan:
    return Plan(plan.goal, plan.steps, True)

def build(plan: Plan) -> BuildArtifact:
    if not plan.approved:
        raise PermissionError("build requires approved plan")
      return  BuildArtifact(["src/module.py",  "tests/test_module.py"],
f"implemented {plan.goal}")
```

這個範例刻意簡化，但包含雙模式架構最重要的約束：未核准的計劃不能進入 build。`create_plan` 產生的是未核准計劃；`approve` 建立一份核准後的新計劃；`build` 在執行前檢查 `approved`。若未核准就呼叫 `build`，程式會拋出 `PermissionError`。

這個設計對應真實多 Agent 系統中的 approval gate。它不只是 UI 上的一個按鈕，而是工作流中的硬性條件。只有當計劃、範圍、風險與驗證方式都被接受後，代理才有權修改檔案或產出 artifacts。

`BuildArtifact` 則代表 Build 階段的交付物。它至少要包含修改檔案清單與摘要。實務上還可以加入測試結果、diff 摘要、風險說明與下一步建議。好的 Build 不是只說「完成」，而是清楚交代完成了什麼。

4.5 與其他 Agent 的交接

OpenCode 不應孤立工作。它的輸入常來自 Hermes 的規格與審稿意見，輸出則交給 Pi 和 Codex 進一步處理。為了讓交接順利，每次 Plan / Build 都應留下可讀摘要。

交給 Hermes 時，摘要應包含：章節目標、核心論點、可能需要技術審查的地方。Hermes 關心的是邏輯、術語一致性與風險。

交給 Pi 時，摘要應包含：修改了哪些 Markdown 段落、是否有特殊格式、是否需要 Typst build。Pi 關心的是精準修補與排版穩定。

交給 Codex 時，摘要應包含：正文引用了哪些程式範例、測試命令是什麼、是否新增或改動 examples。Codex 關心的是可執行性。

交給 Antigravity 或總編排器時，摘要應包含：目前階段是否完成、下一階段能否開始、是否有阻塞或需要人工決策。

4.6 常見錯誤與反模式

第一個反模式是沒有 Plan 直接 Build。這通常會讓代理過早鎖定某個解法，忽略更好的任務拆解。

第二個反模式是 Plan 太空泛。例如「我會改善品質」不是計劃，因為它沒有範圍、步驟與驗證方式。

第三個反模式是 Build 擴張範圍。代理原本只要修改一章，卻順手重構整個目錄。這會破壞多 Agent 分工。

第四個反模式是核准流於形式。如果所有計劃都自動通過，approval gate 就失去意義。真正需要核准的，是會影響大量檔案、刪除資料、引入依賴、改變發布流程的決策。

4.7 驗證策略

執行測試：

```
cd examples/ch04_opencode_workflow
PYTHONPATH=. python3 -m pytest -v
```

測試應確認兩件事：未核准計劃不能 build；核准後的計劃能產生 artifact。這雖然簡單，卻代表整個雙模式架構的核心邊界。

在真實專案中，驗證還應包含：Plan 是否列出明確檔案範圍、Build 是否只修改核准範圍、完成摘要是否足以讓下一個代理接手。

4.8 實戰練習

進行大型任務前，可以直接複製以下 Plan 模板：

```
## 任務目標

## 修改範圍
```

```
## 不修改範圍

## 實作步驟

## 風險

## 驗證方式

## 需要人工核准的動作
```

Build 完成後，建議使用以下回報模板：

```
## 已修改檔案

## 主要變更

## 驗證命令

## 驗證結果

## 後續建議
```

這兩份模板可以降低交接成本，也能讓使用者清楚知道代理是否按照原計劃執行。

1. 為 `Plan` 增加 `risks` 與 `verification` 欄位。
2. 修改 `build`，讓它在缺少驗證步驟時拒絕執行。
3. 為章節擴充任務寫一份 Plan，列出目標、檔案、步驟、風險與驗證。
4. 設計一個 approval policy：哪些任務可自動核准，哪些必須人工核准。
5. 將 BuildArtifact 擴充為包含測試結果與修改摘要。

4.9 實務案例：章節擴充任務如何套用 Plan / Build

以本書擴充章節為例，Plan 模式應先提出一份清楚任務合約。目標可以是「將第 5 章擴充到 3000 字以上，補上 Hermes 記憶與審稿實務」。範圍應限制在 `src/ch05.md`，除非另有核准，不修改 examples、templates 或 scripts。步驟可以是：讀取現有章節、對照 `examples/ch05_hermes_self_learning/memory.py`、補充長程記憶與反思流程、加入反模式與練習、統計字數、執行測試與 build。

風險則包括：新增 Markdown 語法可能造成 Typst build 失敗；正文可能引用不存在的函式；擴充內容可能偏離章節規格；字數增加但教學價值不足。驗證方式是重新統計字數、執行 `scripts/test_all.sh`、執行 `scripts/build.py`。

只有當這些內容被使用者或總編排器接受，Build 模式才開始修改。Build 完成後也不應只回報「已完成」，而要列出修改檔案、主要新增內容、字數變化與驗證結果。這種交接格式能讓 Hermes 接著審稿，也能讓 Pi 進行小型格式修補。

4.10 成本與品質的取捨

Plan / Build 還有一個實務價值：控制 LLM 成本。並非所有任務都需要昂貴模型。Plan 階段若涉及架構決策、風險判斷與跨檔案推理，可以使用品質較高的付費模型；Build 階段若只是依照已核准計劃填充固定格式，則可以使用成本較低的模型或本地模型。最後審稿再交給 Hermes 或高品質模型抽查。

這種分層策略比「所有任務都用最強模型」更可持續，也比「所有任務都用免費模型」更穩定。多 Agent 系統的優勢不只在平行化，也在於能把不同品質與成本的模型配置到最適合的位置。

4.11 Plan / Build 檢查清單

進入 Build 前，建議逐項檢查：目標是否可驗收、檔案範圍是否明確、是否列出不修改項目、是否說明風險、是否定義驗證命令、是否標示需人工核准的動作。若其中任何一項缺失，代理應回到 Plan，而不是勉強進入 Build。

Build 完成後，也應檢查：實際修改是否符合計劃、是否有額外檔案被改動、測試是否通過、是否留下交接摘要、是否有後續待辦。這些檢查可以被做成模板，讓每次大型任務都遵循同樣節奏。

對寫書專案而言，Plan / Build 檢查清單還能避免內容膨脹失控。每章擴充都應確認新增內容服務於學習目標，而不是單純增加字數。好的 Build 不是把章節寫長，而是把讀者需要的背景、實作與判斷補齊。

4.12 你現在應該能做到

讀完本章後，你應該能：

- 寫出一份包含目標、範圍、風險與驗證方式的 Plan。
- 說明 approval gate 為什麼能降低大型修改風險。
- 使用 Build 回報模板交接修改結果。
- 判斷哪些任務適合 Plan，哪些任務可以直接微修。

4.13 本章小結

OpenCode 的 Plan / Build 雙模式把大型任務拆成可審查的決策與可驗證的執行。Plan 保護方向，Build 落實修改，approval gate 則避免代理在不確定時直接動手。

下一章將介紹 Hermes 的長程記憶與反思審稿。當 OpenCode 能產出章節與方案後，團隊還需要一個角色負責跨章一致性、歷史錯誤記憶與品質規則萃取，這正是 Hermes 的核心位置。

第 5 章 長程推理與反思中樞：Hermes Agent 的自主學習

> Hermes 承擔長程推理、決策紀錄與反思回饋。本章實作一個記憶型審查器，讓代理從歷次失敗中提取規則。

5.1 本章導讀：為什麼團隊需要記憶

多 Agent 系統若沒有記憶，就像一支每天失憶的團隊。今天發現某章術語不一致，明天可能又犯一次；今天 build 因字型缺失失敗，下一次仍可能選用不存在的字型；今天測試因 timeout 卡住，下一章仍可能忘記加上 timeout guard。這類問題不是單次模型推理能完全解決的，而是需要長程記憶與反思機制。

Hermes 在本書團隊中扮演技術主編與反思中樞。它不只是閱讀單一章節，而是觀察整個專案的規則、錯誤與決策歷史。當某次審稿發現問題，Hermes 應把它萃取為可重用規則，並在之後的章節、範例或排版流程中提前提醒其他代理。

本章會介紹記憶型審查器的最小模型：觀察事件、萃取規則、提供 prompt hints、根據記憶評估風險。範例位於 `examples/ch05_hermes_self_learning/`。雖然程式很短，但它展示了自主學習系統的核心：不是保存所有文字，而是從經驗中提煉可行動的規則。

```
flowchart LR
    A[審稿觀察] --> B[萃取規則]
    B --> C[長程記憶]
    C --> D[下次任務提示]
    D --> E[降低重複錯誤]
```

5.2 長程記憶在寫書任務中的價值

寫一本科技書不是一次性回答，而是長時間累積。章節之間必須共享術語、語氣、範例命名、目錄結構、測試方式與排版規則。若每一章都從零開始，整本書會像由不同作者拼湊而成：第一章稱為「代理」，第二章稱為「Agent」，第三章又改成「Agent 程式」；某章用 `examples/ch03_*`，另一章卻引用不存在的路徑；某章說測試要用 pytest，另一章沒有說明如何執行。

Hermes 的第一個價值是維持一致性。它可以記住全書採用的術語，例如「多 Agent」、「監管中樞」、「品質閘門」、「微修」。它也能記住章節固定結構，例如本章導讀、核心概念、實戰範例、驗證策略、反模式、練習與小結。

第二個價值是避免重複錯誤。若某次排版失敗是因為 Typst 不支援某個語法，Hermes 應把這件事記為規則，而不是只修當下錯誤。下次代理撰寫 Markdown 或 Typst 時，就能提前避開。

第三個價值是支援跨章推理。第 7 章的出版 pipeline 需要引用前面各章的角色分工；第 8 章 CI/CD 又需要引用測試、build 與 release 規則。沒有長程記憶，代理很容易只看局部，忽略全書整體邏輯。

5.3 Hermes 作為技術主編

技術主編不是只抓錯字。Hermes 要檢查的是「這本書是否可信」。可信度包含多個層面。

第一是技術正確性。正文提到的程式、命令與檔案路徑必須存在，概念解釋不能和範例行為矛盾。

第二是架構一致性。每個 Agent 的職責要前後一致。若前面說 Codex 負責 examples，後面就不應讓 Codex 去重寫正文；若 Pi 被定位為微修代理，就不應描述它主導大型重構。

第三是教學可讀性。章節不應只有抽象概念，也要有情境、實作、反模式與練習，讓讀者能把觀念轉成操作。

第四是出版品質。Markdown 需能轉成 Typst，PDF 要能 build，程式範例要能測試，release artifacts 要有 checksum 與 manifest。

Hermes 的審稿報告最好分級：重大錯誤、結構問題、風格問題、可讀性建議。重大錯誤會阻止進入下一階段；風格建議則可以排入 Pi 的微修任務。

5.4 實戰範例：ReflectionMemory

本章範例位於：

```
examples/ch05_hermes_self_learning/
```

核心程式碼如下：

```
from __future__ import annotations
from dataclasses import dataclass, field
from typing import List

@dataclass
```

```python
class ReflectionMemory:
    observations: List[str] = field(default_factory=list)
    rules: List[str] = field(default_factory=list)

    def observe(self, text: str) -> None:
        self.observations.append(text)
        lowered = text.lower()
        if "timeout" in lowered:
            self.rules.append("add_timeout_guard")
        if "overwrite" in lowered:
            self.rules.append("check_file_ownership")

    def prompt_hints(self) -> List[str]:
        return sorted(set(self.rules))

def review_risk(memory: ReflectionMemory, task: str) -> str:
    hints = memory.prompt_hints()
    if "add_timeout_guard" in hints and "network" in task.lower():
        return "HIGH"
    return "LOW"
```

這段程式展示了反思記憶的最小循環。`observe` 接收觀察文字，例如「network command timeout」或「agent overwrite another file」。它把原始觀察保存起來，並從文字中萃取規則。若看到 timeout，就加入 `add_timeout_guard`；若看到 overwrite，就加入 `check_file_ownership`。

`prompt_hints` 回傳去重後的規則清單，可在下一次任務提示中使用。這很重要：記憶不是為了保存一切，而是為了在正確時機提供可行動提示。

`review_risk` 則示範如何用記憶評估新任務風險。若過去曾因 timeout 出問題，而新任務又涉及 network，就提高風險。真實系統可以更複雜，例如根據任務類型、檔案範圍、歷史失敗次數與代理身份計算風險分數。

5.5 反思迴圈：Review 到 Rule

Hermes 的工作流程可以整理成五步：

1. Review：閱讀產物、測試結果與 build log。
2. Diagnose：判斷問題屬於技術錯誤、結構問題還是風格問題。
3. Extract：把具體問題轉成一般規則。
4. Apply：在下一個任務中把規則注入 prompt 或檢查清單。
5. Verify：觀察同類問題是否減少。

例如，若某章正文引用了不存在的 `examples/ch09`，Hermes 不應只修成正確路徑，也應萃取規則：「正文引用 examples 路徑前，必須檢查檔案存在」。下次審稿時，這條規則就會成為檢查項。

再例如，若 build 因 Markdown 中某些特殊符號造成 Typst 解析失敗，規則可以是：「新增 code block 或特殊符號後必須重新執行 build」。這種規則會直接幫助 Pi 與 OpenCode。

再看一個更貼近本書的完整例子：

```
QA 問題：第 8 章提到 release manifest，但專案中沒有實際打包腳本。
Hermes 判斷：正文描述與專案狀態不一致，屬於技術可信度問題。
萃取規則：正文提到的自動化流程，必須有 scripts/ 中的對應實作或明確標示為未來工作。
後續動作：交給 Antigravity 或 Pi 補 `scripts/package_release.py`，完成後重跑測試、build 與打包流程。
```

這個例子說明，Hermes 的價值不只是指出某句話不夠好，而是把一次錯誤轉成未來可重用的檢查規則。當下一章又提到某個自動化能力時，這條規則就會提醒團隊：文件與專案實作必須同步。

5.6 記憶污染與過度自信

長程記憶也有風險。第一個風險是記憶污染。若 Hermes 從錯誤觀察中萃取錯誤規則，之後會不斷放大偏差。例如某次 timeout 是網路暫時問題，卻被記成「所有網路任務都禁止執行」，就會過度保守。

第二個風險是過度自信。記憶提供的是提示，不是絕對真理。每次任務仍要根據當前上下文判斷。過去某個檔案由 Codex 維護，不代表未來永遠不能由其他代理在核准後修改。

第三個風險是記憶過量。若保存太多細節，prompt 會變得冗長，反而降低效率。Hermes 應定期整理記憶，把重複觀察合併成少數高價值規則。

5.7 驗證策略

執行測試：

```
cd examples/ch05_hermes_self_learning
PYTHONPATH=. python3 -m pytest -v
```

測試應確認：觀察 timeout 會產生 `add_timeout_guard`，觀察 overwrite 會產生 `check_file_ownership`，而帶有 network 的任務在存在 timeout 規則時會被評為高風險。

這些測試確保記憶不是裝飾，而是真的能影響後續判斷。

5.8 實戰練習

1. 為 `ReflectionMemory` 增加 `severity`，區分 warning 與 blocker。
2. 新增一條規則：當觀察到 `typst` 與 `font` 時，加入 `check_available_fonts`。
3. 實作 `review_checklist()`，把規則轉成審稿清單。
4. 設計一份章節審稿報告模板，包含重大錯誤、結構問題、風格建議與下一步。
5. 思考何時應刪除或降權舊記憶，避免記憶污染。

5.9 設計審稿報告格式

為了讓 Hermes 的審稿能被其他 Agent 使用，報告應避免長篇散文式評論，而要採用固定結構。建議格式如下：

```
章節：ch05
結論：PASS / REVISE / BLOCKED
重大錯誤：列出會阻止發布的問題
結構問題：列出章節安排與邏輯問題
技術問題：列出範例、命令、路徑與概念錯誤
風格建議：列出術語、語氣與可讀性修正
可萃取規則：列出應加入長程記憶的規則
下一步：交給哪個 Agent 處理
```

這種格式有兩個好處。第一，總編排器可以直接讀取結論，判斷是否進入下一階段。第二，Pi、OpenCode、Codex 可以根據問題類型接手。例如技術問題交給 Codex，風格建議交給 Pi，結構問題交給 OpenCode，規則萃取則回到 Hermes 記憶。

審稿報告也應避免只有批評而沒有處置建議。好的 Hermes 不只是說「這裡不好」，而是指出「這裡為什麼會影響讀者理解，應交由哪個角色修正，修正後要用什麼方式驗證」。

5.10 人類主編與 Hermes 的關係

Hermes 可以大幅降低審稿成本，但不應完全取代人類主編。人類主編負責最終取捨：書的定位是否正確、章節深度是否符合讀者、某些內容是否應刪減或延伸、出版時機是否成熟。Hermes 則負責把大量細節變成可檢查清單，提醒人類不要漏掉重複錯誤。

在成熟流程中，Hermes 會先做機器審稿，標出高風險問題；OpenCode 或 Pi 修正後，再由 Hermes 複查；最後人類主編只需聚焦在少數策略性問題。這種分工不是降低人的重要性，而是把人的注意力從機械檢查釋放到真正的編輯判斷。

5.11 記憶的生命週期管理

長程記憶不是只增不減。每條規則都應有來源、時間與使用次數。若某條規則長期沒有被觸發，可以降權；若某條規則多次避免錯誤，則應提升為正式檢查項；若某條規則被證明錯誤，應標記為 deprecated，而不是默默保留。

此外，記憶可以分層保存。專案級記憶記錄本書特有規則，例如目錄結構與術語；通用級記憶記錄所有科技書都適用的規則，例如程式碼需可測試；短期記憶記錄當前任務上下文。分層能避免把所有規則都塞進每次 prompt。

5.12 Hermes 審稿檢查清單

每次審稿前，Hermes 可先載入固定清單：章節標題是否符合規格、術語是否一致、是否引用正確範例路徑、程式碼是否有測試、反模式是否足夠具體、練習是否可操作、是否有銜接前後章。這些項目不需要每次重新發明，而應成為審稿記憶的一部分。

審稿後，Hermes 應把新發現分成兩類：一次性問題與可重用規則。一次性問題直接交給對應 Agent 修正；可重用規則則加入記憶，成為下一章的提前提醒。這能讓審稿越來越有效率，而不是每章都從零開始。

5.13 你現在應該能做到

讀完本章後，你應該能：

- 說明 Hermes 如何從審稿觀察萃取可重用規則。
- 區分一次性問題與長程記憶規則。
- 設計一份包含重大錯誤、結構問題與風格建議的 QA 報告。
- 辨識記憶污染與過度自信風險。

5.14 本章小結

Hermes 的價值在於讓多 Agent 團隊不只是完成單次任務，而是能從過去錯誤中學習。透過觀察、萃取規則、注入提示與風險評估，團隊能逐步降低重複錯誤，維持全書一致性。

下一章將進入 Codex 的自動化測試與安全沙盒。Hermes 告訴我們應該檢查什麼，而 Codex 則負責把檢查落實為可執行、可重跑的驗證流程。

第 6 章 程式碼驗證沙盒：Codex 自動化測試與安全隔離

> Codex 的邊界在於可執行驗證。本章以安全沙盒執行測試命令，說明超時、環境隔離與輸出摘要如何降低風險。

6.1 本章導讀：科技書的程式碼必須能跑

很多技術文章最大的問題，是範例看起來合理，實際上不能執行。指令少了一個參數、檔案路徑寫錯、函式名稱和測試不一致、依賴版本沒有說明，讀者照做時就會失敗。對一本實戰型科技書來說，這會直接破壞信任。

Codex 在本書團隊中的核心責任，是把「看起來能用」變成「已被驗證」。它負責 examples 目錄中的程式、測試與自動化驗證。每個章節如果引用程式範例，該範例就應該能在本機或 CI 中重跑。這不只是工程潔癖，而是教學品質的基本要求。

然而，讓代理執行命令也有風險。命令可能卡住、輸出過大、修改環境、刪除檔案，或因網路依賴導致不穩定。因此 Codex 不應只是盲目執行 shell，而要透過沙盒包裝命令，限制 timeout，捕捉 stdout/stderr，保存 return code，並把結果摘要成上層代理能理解的訊號。

本章以 `examples/ch06_sandbox_runner/` 示範最小沙盒執行器。

```
flowchart LR
    A[測試命令] --> B[沙盒執行]
    B --> C[stdout stderr returncode]
    C --> D[PASS FAIL TIMEOUT]
    D --> E[品質閘門]
```

6.2 文件範例與可測試範例

文件範例只要讀起來合理；可測試範例則必須能被機器驗證。兩者差異很大。

文件範例可能省略 import、假設某個變數已存在，或用「概念性偽碼」表達想法。這在概念文章中可以接受，但在實戰書中要特別標明。若讀者以為這段可以直接執行，卻發現無法跑通，就會失去信心。

可測試範例則要求更嚴格。它必須有完整檔案、明確入口、測試案例與執行命令。本書把每章範例放在獨立目錄，例如 `examples/ch06_sandbox_runner/`，並提供 `pyproject.toml` 與 `test_baseline.py`。這讓全書可以用 `scripts/test_all.sh` 一次驗證。

Codex 的工作不是讓每個範例變得龐大，而是讓每個範例都足以驗證本章核心觀念。第 6 章的核心不是做出完整容器平台，而是展示 timeout、return code、stdout/stderr 與摘要判斷，因此範例保持小而可測。

6.3 沙盒設計原則

第一個原則是隔離。測試命令應在明確工作目錄中執行，並盡量避免修改專案外部狀態。若需要臨時檔案，應使用暫存目錄。

第二個原則是 timeout。任何命令都可能卡住，尤其是網路、長時間測試、互動式 prompt 或死循環。沒有 timeout 的自動化系統，遲早會被某個命令拖死。

第三個原則是完整捕捉輸出。stdout 與 stderr 都是診斷資訊。return code 則是機器判斷成功或失敗的基本訊號。沙盒不應只顯示最後一行，也不應吞掉錯誤。

第四個原則是摘要化。上層編排器不一定要閱讀完整 log，它需要先知道結果是 `PASS`、`FAIL` 還是 `TIMEOUT`。若結果不通過，再下鑽查看詳細輸出。

第五個原則是危險命令防護。刪除檔案、改系統設定、安裝全域套件、上傳資料、讀取敏感金鑰等操作都不應自動執行。這些操作需要人工核准或更嚴格沙盒。

6.4 實戰範例：sandbox.py

本章範例位於：

```
examples/ch06_sandbox_runner/
```

核心程式碼如下：

```python
from __future__ import annotations
import subprocess
from dataclasses import dataclass
from typing import List

DENYLIST = {"rm", "sudo", "mkfs", "shutdown"}
```

```
@dataclass(frozen=True)
class SandboxResult:
    returncode: int
    stdout: str
    stderr: str
    timed_out: bool = False

def validate_command(command: List[str]) -> None:
    """Reject commands that are too risky for the teaching sandbox."""
    dangerous = DENYLIST.intersection(command)
    if dangerous:
        joined = ", ".join(sorted(dangerous))
        raise PermissionError(f"dangerous command rejected: {joined}")

def run_command(command: List[str], timeout: float = 2.0) ->
SandboxResult:
    validate_command(command)
    try:
        proc = subprocess.run(command, text=True, capture_output=True,
timeout=timeout, check=False)
        return SandboxResult(proc.returncode, proc.stdout, proc.stderr,
False)
    except subprocess.TimeoutExpired as exc:
        return SandboxResult(124, exc.stdout or "", exc.stderr or "", True)

def summarize(result: SandboxResult) -> str:
    if result.timed_out:
        return "TIMEOUT"
    return "PASS" if result.returncode == 0 else "FAIL"
```

`SandboxResult` 是命令執行結果的資料合約。它沒有只回傳布林值，因為失敗有很多種：return code 非零、timeout、stderr 有錯誤訊息、stdout 顯示測試失敗。保留完整資訊可以讓 Hermes 或人類主編做後續診斷。

`validate_command` 是第一道安全閘門。它用 denylist 拒絕 `rm`、`sudo`、`mkfs`、`shutdown` 等高風險命令。這不是完整資安沙盒，但能讓讀者看見一個重要原則：命令應先被分類與核准，再被執行。

`run_command` 使用 `subprocess.run`，設定 `capture_output=True` 與 timeout。`check=False` 很重要，因為我們不希望 Python 直接丟例外中斷流程；我們希望把失敗轉成結構化結果，交給上層決策。

當 timeout 發生時，函式回傳 return code `124`，這是許多命令列工具慣用的 timeout 程式碼。`timed_out=True` 則讓摘要函式不必猜測失敗原因。

`summarize` 把結果壓縮成三種狀態。這正是品質閘門需要的輸入：PASS 可以進入下一階段；FAIL 需要修正；TIMEOUT 可能需要調整 timeout、檢查死循環或改用更隔離的環境。

6.5 在全書 CI 中使用 Codex 驗證

本書的 `scripts/test_all.sh` 會逐一進入各章 examples 目錄執行 pytest。這種設計有幾個好處。

第一，章節範例彼此隔離。第 3 章的 micro patch 測試不應依賴第 6 章的 sandbox runner。

第二，失敗容易定位。若 `ch06_sandbox_runner` 失敗，維護者可以直接進入該目錄重跑測試。

第三，書籍 build 與程式測試可以分開。文字與排版由 Typst 驗證，程式行為由 pytest 驗證，最後再由出版 pipeline 串起來。

第四，讀者也能學習專案組織方式。每章都有 README、程式、測試與 pyproject，這比把所有範例塞在一個資料夾更清楚。

6.6 常見錯誤與反模式

第一個反模式是沒有 timeout。只要一個測試卡住，整條 pipeline 就會停住。

第二個反模式是只看 return code，不保留 stdout/stderr。這會讓失敗難以診斷。

第三個反模式是在測試中依賴外部網路。網路測試容易不穩定，若必須使用，應標記為 integration test，並設定嚴格 timeout。

第四個反模式是讓測試修改共享環境。範例測試應盡量使用暫存資料，避免污染其他章節。

第五個反模式是測試與正文脫節。正文提到的函式與命令若沒有被測試覆蓋，很容易在後續修改中失效。

6.7 驗證策略

執行測試：

```
cd examples/ch06_sandbox_runner
PYTHONPATH=. python3 -m pytest -v
```

測試應確認成功命令會得到 `PASS`，失敗命令會得到 `FAIL`，timeout 會得到 `TIMEOUT`，而危險命令會在執行前被 `validate_command` 拒絕。若繼續擴充範例，建議加入工作目錄限制與更細緻的組合命令規則。

6.8 實戰練習

1. 為 `run_command` 增加 `cwd` 參數，限制命令工作目錄。
2. 擴充 `denylist`，支援 `rm -rf` 這類組合規則。
3. 將 stdout/stderr 截斷到固定長度，避免 log 過大。
4. 為 `validate_command` 增加更清楚的錯誤分類。
5. 把 sandbox runner 整合到章節測試腳本中，讓所有命令都經過同一層包裝。

6.9 從本機沙盒到 CI 執行環境

本章的範例使用 Python `subprocess` 建立最小沙盒，但在真實團隊中，執行環境通常會再分成幾個層級。第一層是本機沙盒，適合快速驗證單一命令。第二層是專案虛擬環境，透過固定 Python 版本與依賴檔避免「我這裡可以跑」的問題。第三層是容器或 CI runner，讓測試在乾淨環境中重跑。第四層才是正式發布流程。

多 Agent 團隊應該讓 Codex 優先在最小安全環境中驗證。若單章測試能在本機快速通過，就不需要每次都啟動昂貴容器。若任務涉及系統套件、網路服務或多程序互動，才提高隔離層級。隔離越強，成本越高；成本越高，越需要明確觸發條件。

CI 中的沙盒還需要處理 reproducibility。所有依賴應記錄在 `pyproject.toml`、`requirements.txt` 或 lock file 中；測試不應依賴執行順序；需要時間的測試應避免用真實等待，而是用 mock 或縮短 timeout；產生的臨時檔案應在測試結束後清理。

6.10 危險命令與人工核准

自動化測試並不代表任何命令都能自動執行。多 Agent 系統應該把命令分成三類。

第一類是安全命令，例如讀檔、列目錄、執行單元測試、編譯本地文件。這類命令可以自動執行。

第二類是需監控命令，例如下載依賴、啟動服務、長時間 build。這類命令可自動執行，但必須有 timeout、log 與取消機制。

第三類是高風險命令，例如刪除檔案、修改系統設定、使用 `sudo`、上傳資料、改動金鑰或發布產物。這類命令必須人工核准。即使代理聲稱必要，也不應跳過審批。

這種分類可以寫成 policy，讓 Codex 在執行前先判斷命令風險。若命令屬於高風險，沙盒不應執行，而應回傳 `REQUIRES_APPROVAL`。這比執行後再補救安全得多。

6.11 測試摘要如何回饋給其他 Agent

Codex 的測試結果不應只停留在終端。它應該回饋給 OpenCode、Hermes、Pi 與總編排器。OpenCode 需要知道哪些範例可以引用；Hermes 需要知道失敗是否影響技術正確性；

Pi 需要知道是否有格式修改導致 build 失敗；Orchestrator 需要決定 pipeline 是否能進入下一階段。

一份好的測試摘要應包含：測試命令、工作目錄、通過數、失敗數、timeout 狀態、關鍵錯誤訊息與下一步建議。若測試全部通過，摘要也要列出執行時間與覆蓋範圍，避免「通過」變成空泛字眼。

在本書專案中，`scripts/test_all.sh` 已提供全書測試總覽；未來可以進一步讓它輸出 JSON 報告，供 Herdr dashboard 或 release pipeline 讀取。如此一來，測試結果就能成為真正的機器可讀品質閘門。

6.12 Codex 驗證檢查清單

Codex 在回報通過前，應確認測試命令可重跑、工作目錄正確、timeout 已設定、stdout/stderr 已保留、失敗訊息足以診斷、測試沒有依賴外部不穩定服務。若範例是教學用途，還要確認 README 中的執行方式與測試命令一致。

若測試失敗，Codex 的任務不是只貼出完整 log，而是先摘要失敗類型：語法錯誤、斷言失敗、timeout、依賴缺失、路徑錯誤或權限問題。這種分類能讓 OpenCode、Pi 或 Hermes 快速接手。

6.13 你現在應該能做到

讀完本章後，你應該能：

- 使用 SandboxResult 保存 return code、stdout、stderr 與 timeout 狀態。
- 用 `summarize` 將命令結果摘要為 PASS、FAIL 或 TIMEOUT。
- 用 `validate_command` 在執行前拒絕高風險命令。
- 說明測試摘要如何回饋給其他 Agent 與品質閘門。

6.14 本章小結

Codex 的核心價值是把技術書中的程式範例變成可執行、可重跑、可驗證的工程資產。沙盒執行器提供 timeout、輸出捕捉與摘要狀態，讓自動化測試既有用又可控。

下一章將把前面幾章的角色串起來，建立完整出版 pipeline：從規格、範例、正文、審稿到 build，讓多 Agent 團隊真正產出一本可發布的書。

第 7 章 全流程編排實戰：用多 Agent 自動化產出科技書

> 多 Agent 出版流水線把規格、範例、正文、審稿與建置串成可重跑流程。本章實作出版任務圖與品質閘門。

7.1 本章導讀：把分工變成流水線

前面幾章分別介紹了 Herdr 的監管、Pi 的微修、OpenCode 的 Plan / Build、Hermes 的反思審稿，以及 Codex 的測試沙盒。這些能力若各自存在，只是一組好工具；真正產生價值的，是把它們串成可重跑的出版流水線。

科技書生產不是單一步驟。使用者提出主題後，需要先形成 PRD，再拆章節規格，接著開發可執行範例，然後撰寫正文，之後審稿、校對、排版、建置、打包與發布。每個階段都有輸入、輸出、負責 Agent 與品質閘門。若沒有編排，整個流程就會依賴人類記憶；若有清楚 pipeline，代理團隊就能穩定交付。

本章以 `examples/ch07_book_pipeline/` 的任務圖為例，說明如何定義出版階段、階段依賴與品質閘門。重點不是做出龐大的 workflow engine，而是理解一個原則：任何可重複的出版工作，都應該被建模為明確階段與可驗證條件。

```
flowchart LR
    A[spec] --> B[example]
    B --> C[draft]
    C --> D[review]
    D --> E[build]
    E --> F[release]
```

7.2 出版 pipeline 的基本階段

本書採用的核心流程可以拆成八個階段。

第一是需求與 PRD。這一階段決定書名、讀者、章節範圍、技術深度、出版格式與成功標準。若 PRD 不清楚，後面所有章節都會漂移。

第二是章節規格。每章要有學習目標、核心概念、範例方向、驗收標準與風險說明。Hermes 適合維護這一層，因為它關注全書一致性。

第三是程式範例。Codex 根據規格建立 examples，寫測試並確保可執行。正文應引用已驗證的範例，而不是先寫概念再希望程式能補上。

第四是正文草稿。OpenCode 根據規格與範例撰寫章節，把程式、概念與情境整合成讀者能理解的教學內容。

第五是審稿。Hermes 檢查技術正確性、跨章一致性與教學完整度，產出 QA 報告。

第六是微修與格式。Pi 修正標點、Markdown 結構、Typst 相容性與小型文字問題。

第七是建置。Antigravity 或 build script 將 Markdown 轉成 Typst，產出 cover 與 interior PDF。

第八是發布。產生 manifest、checksum、release note 與上架素材，準備交付讀者或平台。

7.3 階段輸入、輸出與負責 Agent

每個階段都應有明確輸入與輸出。沒有輸入，代理會亂猜；沒有輸出，下一階段無法驗收。

PRD 階段的輸入是使用者需求，輸出是 `docs/prd.md`。章節規格階段的輸入是 PRD，輸出是 `docs/chapter_specs/chXX_spec.md`。範例階段的輸入是章節規格，輸出是 `examples/chXX_*/` 以及測試結果。正文階段的輸入是規格與範例，輸出是 `src/chXX.md`。審稿階段的輸入是正文與測試結果，輸出是 `qa/reports/chXX_review_report.md`。建置階段的輸入是所有正文與模板，輸出是 `dist/cover.pdf` 與 `dist/book_interior.pdf`。

這種設計有助於代理交接。OpenCode 不必猜測範例在哪裡；Codex 不必理解整本書所有排版；Pi 不必重新審查技術架構；Hermes 不必執行所有命令。每個 Agent 都在明確邊界中貢獻。

階段	輸入	輸出	負責 Agent	驗收
spec	使用者需求 / PRD	`docs/chapter_specs/`	Hermes	規格完整、目標清楚
example	章節規格	`examples/chXX_*/`	Codex	pytest 通過
draft	規格 + 範例	`src/chXX.md`	OpenCode	章節完成、引用正確
review	正文 + 測試結果	`qa/reports/`	Hermes	QA 通過、無重大錯誤
build	`src/` + `templates/`	`dist/*.pdf`	Pi / Antigravity	Typst build 通過
release	PDF + docs + QA	manifest + zip	Antigravity	checksum 完整、人工核准

這張表是全書生產流程的總覽。當任務卡住時，先判斷它屬於哪個階段，再回到該階段的輸入、輸出與負責 Agent，而不是讓所有代理同時介入。

7.4 實戰範例：pipeline.py

本章範例位於：

```
examples/ch07_book_pipeline/
```

核心程式碼如下：

```
from __future__ import annotations
from dataclasses import dataclass
from typing import Dict, List

@dataclass(frozen=True)
class Stage:
    name: str
    requires: List[str]

def default_stages() -> List[Stage]:
    return [
        Stage("spec", []),
        Stage("example", ["spec"]),
        Stage("draft", ["example"]),
        Stage("review", ["draft"]),
        Stage("build", ["review"]),
        Stage("release", ["build"]),
    ]

def ready_stages(done: List[str], stages: List[Stage]) -> List[str]:
    completed = set(done)
     return [s.name for s in stages if s.name not in completed and
set(s.requires) <= completed]

def quality_gate(results: Dict[str, bool]) -> bool:
    return all(results.get(key, False) for key in ["tests", "review",
"build"])
```

`Stage` 定義了階段名稱與依賴。`default_stages` 建立一條簡化出版流程：先規格、再範例、再草稿、再審稿、再建置，最後進入 release。`ready_stages` 根據已完成階段計算下一步可執行項目。這讓 pipeline 不必靠硬編碼順序，而能根據依賴判斷。

`quality_gate` 則是發布前的最低檢查：測試、審稿與 build 都必須通過。真實出版流程可以加入更多條件，例如封面存在、字數達標、checksum 產生、release note 完成、人工最終核准。

7.5 品質閘門：不要把半成品推到下一階段

品質閘門是 pipeline 的核心。沒有品質閘門，流程只會變成自動搬運半成品。

規格階段的閘門可以是：章節目標明確、範例方向明確、驗收方式明確。範例階段的閘門是測試通過。正文階段的閘門是引用路徑存在、內容達到字數與結構要求。審稿階段的閘門是無重大錯誤。建置階段的閘門是 PDF 成功產出。發布階段的閘門是 artifacts、manifest 與 checksum 完整。

品質閘門也能防止代理互相放大錯誤。如果 Codex 的範例沒有通過測試，OpenCode 不應引用它寫教學；如果 Hermes 指出章節技術錯誤，Pi 不應只修標點就進入 build；如果 build 失敗，release 不應繼續打包。

7.6 衝突處理與交接格式

多 Agent 協作一定會遇到衝突。常見衝突包括：規格要求 A，但範例實作成 B；正文引用舊函式名稱；審稿要求重寫，但 build 已經進入排版；Pi 修格式時發現原文邏輯不通。

處理衝突的原則是回到任務合約。誰擁有該檔案？哪個階段是權威來源？目前品質閘門是否通過？例如程式行為以 examples 測試為準，章節意圖以 chapter_specs 為準，出版產物以 build 結果為準。

交接摘要應包含五項：目前階段、已完成產物、待處理問題、驗證結果、下一步建議。這樣下一個 Agent 不必讀完整歷史，也能安全接手。

7.7 本專案目錄結構解析

本書專案的目錄本身就是 pipeline 的落地形式。

`docs/` 保存 PRD 與章節規格，代表需求與設計層。`examples/` 保存可執行範例，代表工程驗證層。`src/` 保存正文，代表教學內容層。`qa/reports/` 保存審稿報告，代表品質記錄層。`templates/` 保存 Typst 樣式，代表排版層。`scripts/` 保存測試與建置腳本，代表自動化層。`dist/` 保存產物，代表發布輸出層。

這種結構讓代理分工自然落地。Codex 修改 examples，OpenCode 修改 src，Hermes 修改 docs 與 qa，Pi 修正文與模板的小型問題，Antigravity 編排 scripts 與 dist。清楚的目錄責任制可以降低互相覆蓋的風險。

7.8 驗證策略

執行測試：

```
cd examples/ch07_book_pipeline
PYTHONPATH=. python3 -m pytest -v
```

測試應確認：在沒有完成任何階段時，只有 `spec` ready；完成 `spec` 後，`example` ready；品質閘門必須在 tests、review、build 都為真時才通過。

全書層級還應執行：

```
bash scripts/test_all.sh
python3 scripts/build.py
```

這代表 pipeline 不只在單章概念上成立，也能支撐整本書產物。

7.9 實戰練習

1. 在 `default_stages` 中加入 `package` 階段，放在 `build` 與 `release` 之間。
2. 為 `quality_gate` 增加 `checksum` 與 `release_note` 條件。
3. 設計一份 handoff 模板，讓每個 Agent 交接時填寫。
4. 為本書新增一個 `status.md`，記錄各章目前階段。
5. 思考若兩個 Agent 同時想修改同一章，應由誰決定合併順序。

7.10 版本控管與 Release Candidate

當 pipeline 能穩定產出 PDF 後，下一個問題是版本控管。書籍和軟體一樣需要版本。草稿階段可以使用 `0.x`，首次可公開版本可以標為 `1.0.0`，小幅修正可以增加 patch 版本，大幅新增章節則增加 minor 版本。

Release Candidate，簡稱 RC，是正式發布前的候選版本。RC 的意義是：內容、範例、測試與排版都已完成，但仍保留最後審查時間。多 Agent 團隊可以為 RC 設計額外檢查，例如全書字數達標、所有 QA 報告為 PASS、PDF 頁面無明顯排版錯誤、release manifest 完整。

版本控管也能幫助讀者。若讀者回報第 6 章範例有問題，維護者可以詢問版本號與 checksum，確認問題發生在哪次發布。沒有版本資訊的書籍，很難進行可靠支援。

7.11 Pipeline 的漸進式自動化

出版 pipeline 不必一開始就完全自動化。建議採用漸進式策略。第一階段先自動化測試與 build；第二階段加入字數統計、路徑檢查與 QA 報告彙整；第三階段加入 package 與 manifest；第四階段才考慮自動上傳或上架。

漸進式自動化的好處是容易定位風險。若一開始就把所有步驟綁在一起，失敗時很難判斷是範例問題、正文問題、排版問題還是發布問題。每增加一層自動化，都應先確保前一層穩定。

這也符合多 Agent 協作精神：先建立可觀測狀態，再建立可驗證交付，最後才增加自動決策。自動化不是越多越好，而是每一步都要能被解釋、被測試、被停止。

7.12 度量指標：如何知道書越來越完整

Pipeline 還應提供度量指標。對書籍來說，常見指標包括：各章字數、範例測試通過率、QA 報告狀態、build 成功率、PDF 產物大小、未解決 TODO 數量、章節規格覆蓋率。

這些指標讓團隊不必憑感覺判斷進度。例如目前若第 2 章有 4000 字、第 3 到第 8 章約 3000 字，就能知道第一輪擴充接近完成；若所有 tests 通過但 QA 報告尚未更新，就知道下一步是審稿而不是繼續寫程式。

7.13 出版流水線檢查清單

一次完整出版任務應確認：PRD 已存在、章節規格已存在、examples 測試通過、正文已達最低篇幅、QA 報告已建立、Typst build 成功、dist 產物存在、release manifest 可產生。這些條件不必全靠人類記憶，可以寫入 pipeline 或狀態報告。

若其中某項未完成，pipeline 應回到對應階段，而不是繼續向前。例如測試未通過就回到 Codex，正文不足就回到 OpenCode，審稿問題就回到 Hermes，排版錯誤就交給 Pi 或 build 維護者。流程能回退，才是真正可用的流程。

7.14 你現在應該能做到

讀完本章後，你應該能：

- 用 Stage 與依賴關係描述出版 pipeline。
- 判斷每個階段的輸入、輸出、負責 Agent 與驗收條件。
- 使用 quality gate 決定是否能進入發布階段。
- 設計一份簡單的專案狀態表與 release candidate 流程。

7.15 本章小結

本章把多 Agent 工具串成完整出版 pipeline。規格、範例、正文、審稿、建置與發布不再是零散動作，而是有依賴、有責任、有品質閘門的流程。

下一章將進入最後一哩路：Typst 排版、封面、release manifest、checksum 與 CI/CD。當內容與測試都準備好後，我們還需要把書變成可交付的出版品。

第 8 章 排版、封面生成與發布 CI/CD

> 最後一章把 Typst 排版、封面資產、版本標記與發行清單整合成 CI/CD 流程，使書籍可持續交付。

8.1 本章導讀：從初稿到可交付出版品

一本書的內容寫完，不代表出版完成。對讀者而言，最終拿到的是 PDF、EPUB、封面圖、書介、版本資訊與下載連結；對作者而言，還需要確保產物可重現、可驗證、可追蹤。若每次上架都靠手動拖檔案、手動改檔名、手動檢查版本，很容易出錯。

第 8 章處理的是最後一哩路：排版、封面、建置、打包與發布。這一階段看似只是「把 Markdown 變 PDF」，實際上包含許多品質細節：頁面尺寸、字型、標題層級、code block、引用框、封面視覺、checksum、release manifest、版本號與人工核准。

本書使用 Typst 作為排版工具，透過 `scripts/build.py` 將 `src/ch*.md` 轉換為 `dist/book_interior.typ`，再編譯成 `dist/book_interior.pdf`；封面則由 `templates/cover.typ` 產出 `dist/cover.pdf`。本章範例 `examples/ch08_publishing_cicd/` 則示範 release manifest 與 checksum 的最小模型。

```
flowchart LR
    A[Markdown 章節] --> B[Typst 轉換]
    B --> C[PDF Build]
    C --> D[Checksum]
    D --> E[Release Manifest]
    E --> F[上架交付]
```

8.2 Markdown 到 Typst 的轉換流程

Markdown 適合寫作，Typst 適合精密排版。兩者的分工很自然：作者與章節代理在 Markdown 中撰寫內容，排版腳本再把標題、列表、引用與程式碼區塊轉成 Typst 語法。

本書的 `scripts/build.py` 負責這件事。它會讀取 `src/` 下所有 `ch*.md`，依檔名排序後逐章轉換，最後合併成 `dist/book_interior.typ`。接著呼叫 `typst compile` 產出 PDF。

這種流程的好處是可重跑。只要來源 Markdown、模板與腳本固定，任何人都可以重新 build 出同樣結構的書籍。這比手動在文書軟體中調整版面更適合多 Agent 協作，因為所有變更都能被 diff、review 與測試。

不過轉換器也有責任邊界。它不需要支援所有 Markdown 擴充語法，但應穩定支援本書使用的語法：標題、段落、列表、引用、程式碼區塊與分隔線。若章節作者引入太特殊的語法，就可能造成 Typst build 失敗。因此 Pi 在微修階段要特別注意 Markdown 是否符合轉換器能力。

8.3 封面設計與內文排版

封面不是裝飾而已，它決定讀者第一眼如何理解書籍定位。科技書封面至少要清楚呈現書名、副標、技術主題與版本感。本書封面以深色科技背景、英文技術標語、中文主標與協作團隊署名構成，對應「多 Agent 系統架構與實戰」的定位。

內文排版則關心可讀性。頁面尺寸採用 ISO-B5，適合技術書閱讀與列印。標題需要有清楚層級，正文需要適當行距，code block 要使用等寬字型，引用框要和一般段落區分。這些設定集中在 `templates/book.typ`，讓章節作者不必每次手動調格式。

字型是排版中常見的坑。若模板指定不存在的字型，Typst 可能警告或 fallback。這就是為什麼排版腳本需要實際 build，而不是只檢查檔案存在。先前本專案曾出現 code block 字型 warning，修正方式是改用系統可用的 `DejaVu Sans Mono`。這是一個典型 Pi 微修任務。

8.4 Release manifest 與 checksum

當 PDF 產出後，還需要知道「這次發布包含哪些檔案」以及「檔案是否被改過」。Release manifest 與 checksum 就是為了解決這個問題。

本章範例位於：

```
examples/ch08_publishing_cicd/
```

核心程式碼如下：

```python
from __future__ import annotations
from dataclasses import dataclass
from hashlib import sha256
from typing import Dict, List

@dataclass(frozen=True)
class ReleaseManifest:
    version: str
    artifacts: Dict[str, str]
```

```
    checks: List[str]

def checksum(data: bytes) -> str:
    return sha256(data).hexdigest()[:12]

def create_manifest(version: str, files: Dict[str, bytes]) ->
ReleaseManifest:
        artifacts = {name: checksum(content) for name, content in
sorted(files.items())}
    checks = ["typst_compile", "artifact_checksum", "human_release_note"]
    return ReleaseManifest(version, artifacts, checks)

def is_publishable(manifest: ReleaseManifest) -> bool:
          return bool(manifest.version and manifest.artifacts and
len(manifest.checks) >= 3)
```

`checksum` 會對檔案內容產生短雜湊。完整 SHA-256 很長，範例取前 12 碼作為教學展示。真實發布可以保存完整雜湊。

`create_manifest` 根據版本號與檔案內容建立 manifest。它記錄 artifacts 名稱與 checksum，也記錄發布前檢查項目。`is_publishable` 則確認 manifest 至少包含版本、產物與基本檢查。

這個模型可以擴充為真實 release metadata，例如：作者、日期、git commit、PDF 頁數、檔案大小、目標平台、授權、上架狀態與人工核准者。

8.4.1 實際打包本書

本專案已提供實際打包腳本：

```
python3 scripts/package_release.py --version 1.0.0
```

執行後會產出：

```
dist/release_manifest.json
dist/multi-agent-book-v1.0.0.zip
```

`release_manifest.json` 會記錄書名、版本、建立時間、artifacts、檔案大小、完整 SHA-256、發布檢查項目與注意事項。這比章節範例中的短 checksum 更接近真實發布需求。

讀者可以打開 manifest 觀察以下欄位：

- `title`：書名。
- `version`：發布版本。

- `created_at`：manifest 建立時間。
- `artifacts`：每個產物的路徑、大小與 SHA-256。
- `checks`：發布前已完成或需確認的檢查。
- `notes`：發布備註，包含人工核准提醒。

需要特別注意的是：package script 只負責打包，不負責公開上架。公開上傳到商店、網站或讀者平台仍應保留人工最終核准。

8.5 CI/CD 流程設計

出版 CI/CD 可以拆成四個階段。

第一是 test。執行 `scripts/test_all.sh`，確保所有 examples 測試通過。這一步由 Codex 的驗證哲學支撐。

第二是 build。執行 `scripts/build.py`，產出封面與內文 PDF。這一步由 Pi 與 Antigravity 共同維護模板與腳本穩定性。

第三是 package。收集 `dist/cover.pdf`、`dist/book_interior.pdf`、README、release note 與 manifest，打包成可發布目錄或壓縮檔。

第四是 release。將 artifacts 上傳到指定平台，或至少產生人工上架所需素材。正式出版時，這一步通常需要人工核准，因為它涉及公開發布與商業資訊。

一個保守的自動化策略是：test、build、package 可以自動執行；真正上架或刪除舊版本必須人工核准。這符合使用者偏好：刪檔案等重大決定外可自動核准，但發布仍應保留最後確認。

8.6 上架素材清單

除了 PDF，本書還需要以下素材：

- 書名與副標。
- 短書介與長書介。
- 作者或團隊介紹。
- 封面圖或封面 PDF。
- 內文 PDF。
- 關鍵字，例如 AI Agent、多 Agent、Herdr、OpenCode、Codex、Typst。
- 版本號與更新紀錄。
- 範例程式下載連結。
- 授權與讀者支援方式。

多 Agent 團隊可以分工準備這些素材。OpenCode 草擬書介，Hermes 檢查技術描述是否誇大，Pi 修文案與標點，Antigravity 負責打包與上傳，Codex 驗證 release manifest。

8.7 常見失敗與處理方式

第一種失敗是 Typst 語法錯誤。通常來自 Markdown 轉換器未處理某些特殊符號，或模板語法不相容。處理方式是定位 `dist/book_interior.typ` 的錯誤行，再回到來源章節或轉換器修正。

第二種失敗是字型缺失。處理方式是列出可用字型，改用系統存在的字型，或把字型納入專案資產。

第三種失敗是章節格式不一致。例如某章缺少一級標題，或 code fence 沒有關閉。這類問題適合由 Pi 寫檢查規則。

第四種失敗是 artifact 遺漏。build 成功但 package 沒有包含封面、manifest 或 release note。這類問題應由 release checklist 防止。

第五種失敗是版本混亂。若檔名、manifest 與 release note 中的版本不一致，讀者會無法確認自己下載的是哪一版。版本號應集中管理。

8.8 驗證策略

執行本章測試：

```
cd examples/ch08_publishing_cicd
PYTHONPATH=. python3 -m pytest -v
```

執行全書建置：

```
bash scripts/test_all.sh
PATH="$PWD/bin:$PATH" python3 scripts/build.py
```

發布前至少確認：測試通過、PDF 產出、manifest 存在、checksum 可重算、release note 已人工審查、上架素材完整。

8.9 實戰練習

1. 為 `ReleaseManifest` 增加 `created_at` 與 `commit` 欄位。
2. 將 `checksum` 改成保存完整 SHA-256。
3. 閱讀 `scripts/package_release.py`，確認 ZIP 中包含哪些 artifacts。
4. 為 `scripts/build.py` 增加頁數或檔案大小摘要。
5. 設計一份上架 checklist，區分自動檢查與人工確認。

8.10 產物檢查清單

正式發布前，建議使用固定清單檢查產物。內容面應確認每章有導讀、核心概念、實作範例、驗證策略、反模式、練習與小結。程式面應確認所有 examples 測試通過，且正文引用的路徑存在。排版面應確認封面與內文 PDF 都能產出，code block 沒有嚴重溢出，標題層級一致。發布面則要確認版本號、manifest、checksum、release note 與上架素材完整。

這份清單可以由 Hermes 維護，Codex 驗證程式項目，Pi 檢查格式項目，Antigravity 負責打包項目。當清單中的項目都通過後，才進入人工最終核准。

8.11 EPUB 與多格式輸出的延伸

本書目前以 PDF 為主要產物，但真實出版常需要 EPUB、HTML 或線上文件。多格式輸出會帶來新的挑戰：PDF 重視固定版面，EPUB 重視可重排文字，HTML 重視連結與響應式閱讀。某些 Typst 設計在 PDF 中很好看，轉到 EPUB 可能不適用。

因此，若未來要支援 EPUB，建議保留 Markdown 作為語意來源，再分別輸出到 Typst/PDF 與 EPUB pipeline。不要把所有資訊鎖死在單一排版格式中。這也是為什麼本書使用 `src/*.md` 作為正文來源，而不是直接在 Typst 中寫完整內容。

8.12 發布後的回饋迴圈

出版不是終點，而是下一輪迭代的開始。讀者回報錯字、範例失效、概念不清或平台相容問題時，應被整理成 issue 或 QA 記錄。Hermes 從回饋中萃取規則，OpenCode 修改正文，Codex 補測試，Pi 做微修，Antigravity 重新打包發布。

這樣一來，書籍就不再是一次性交付，而是可維護產品。多 Agent 團隊的優勢也會在發布後更明顯：它可以快速處理大量小問題，同時保留測試與排版品質。

8.13 發布檢查清單

發布包不只是壓縮檔，而是作者對讀者的交付承諾。manifest 讓讀者與維護者知道自己拿到的是哪個版本；checksum 讓團隊確認檔案未被更動；release checklist 則讓所有人知道哪些步驟已完成、哪些仍需人工確認。當書籍被視為可維護產品時，發布包就是內容、程式碼、審稿與排版之間的責任邊界。

| Artifact | 用途 | 驗收方式 | |—|—|—| | `cover.pdf` | 印刷或 PDF 封面 | 檔案存在、視覺人工確認 | | `book_interior.pdf` | 書籍內文 | Typst build 通過、版面抽查 | | `cover_1600x2400.jpg` | 上架封面圖 | 尺寸符合平台需求 | | `release_manifest.json` | 版本與 checksum 記錄 | SHA-256 可重算 | | `multi-agent-book-v1.0.0.zip` | 發布包 | 包含 PDF、文件、QA 與 manifest |

發布前最後檢查應包含：封面 PDF 存在、內文 PDF 存在、版本號正確、checksum 可重算、release note 已寫、上架文案已審查、範例程式測試通過、授權與作者資訊清楚。若任何一項缺失，都不應進入公開發布。

對商業出版而言，還要加入人工檢查：封面是否符合品牌、書介是否沒有過度承諾、價格與平台欄位是否正確、下載檔是否為最新版本。自動化可以降低錯誤，但最後公開動作仍值得保留人工確認。

8.14 你現在應該能做到

讀完本章後，你應該能：

- 執行 `scripts/build.py` 產出封面與內文 PDF。
- 執行 `scripts/package_release.py` 產生 manifest 與 ZIP 發布包。
- 解讀 release manifest 中的 artifacts、size 與 SHA-256。
- 說明為什麼公開上架仍需人工最終核准。

8.15 全書總結

本書從多 Agent 協作的新範式開始，依序建立了監管中樞、微修代理、Plan / Build 工作流、反思記憶、自動化測試沙盒、出版 pipeline 與 CI/CD 發布流程。每一章都對應一個可執行範例，讓讀者不只理解概念，也能看到最小實作。

真正的多 Agent 系統不是讓更多模型同時說話，而是讓不同能力在清楚邊界、狀態、權限與品質閘門中協作。Herdr 讓狀態可觀測，Pi 讓修改可審查，OpenCode 讓大型任務可規劃，Hermes 讓團隊可反思，Codex 讓範例可驗證，Antigravity 讓流程可編排。當這些角色被串起來，科技書生產就從一次性創作變成可持續交付的工程系統。

完成本章後，讀者可以回到自己的專案，嘗試把任何長程知識工作拆成同樣模式：明確規格、可執行範例、受控寫作、審稿記憶、測試驗證、排版建置與發布清單。這也是本書最重要的實戰精神。

附錄：全書命令速查與發布檢查

本附錄整理本書專案常用命令、Agent 分工與 v1.0 發布檢查清單，方便讀者在實作時快速查閱。

A.1 常用命令速查

檢查 Python 與 pytest：

```
python3 --version
python3 -m pytest --version
```

執行全書範例測試：

```
bash scripts/test_all.sh
```

只執行單章測試，例如第 6 章：

```
cd examples/ch06_sandbox_runner
PYTHONPATH=. python3 -m pytest -v
```

建置封面與內文 PDF：

```
PATH="$PWD/bin:$PATH" python3 scripts/build.py
```

打包 release artifacts：

```
python3 scripts/package_release.py
```

檢查產物：

```
ls -lh dist/
```

A.2 Agent 分工速查

| Agent | 主要責任 | 主要目錄 | 驗收方式 | |—|—|—|—| | Herdr | 終端多工、狀態監管、人工接管提示 | 外部監管層 / dashboard | 狀態正確、阻塞可見 | | Pi | 微修、標點、Markdown / Typst 精修 | `src/`, `templates/` | diff 小、build 通過 | | OpenCode | Plan / Build、章節主筆、任務合約 | `src/` | 章節符合規格 | | Hermes | 技術主編、長程記憶、QA 審稿 | `docs/`, `qa/reports/` | QA 報告通過 | | Codex | 範例程式碼、測試、沙盒驗證 | `examples/`, `scripts/test_all.sh` | pytest 通過 | | Antigravity | 總編排、發布、封面與 CI/CD | `scripts/`, `dist/`, `assets/` | release artifacts 完整 |

A.3 v1.0 發布檢查清單

內容面：

- [x] 8 章正文已完成。
- [x] 前言與附錄已建立。
- [x] 每章都有對應 examples 目錄。
- [x] 每章 QA 報告已更新。
- [x] 術語已依 `docs/style_guide.md` 統一。

程式面：

- [x] `scripts/test_all.sh` 全部通過。
- [x] 每章 pytest 可獨立執行。
- [x] 範例使用 Python 標準庫為主，降低安裝負擔。

排版面：

- [x] `scripts/build.py` 可產出 `dist/cover.pdf`。
- [x] `scripts/build.py` 可產出 `dist/book_interior.pdf`。
- [x] Typst 字型警告已修正。
- [x] 封面 JPG 已產出，可作為上架素材。

發布面：

- [x] `scripts/package_release.py` 可產生 release manifest。
- [x] release zip 包含 PDF、封面、README、QA 與規格文件。
- [x] checksum 可用於確認 artifacts 未被竄改。
- [ ] 正式公開上架前，仍需人工最終核准。

A.4 常見問題排除

若 pytest 找不到模組，請確認在章節 examples 目錄中執行，並設定：

```
PYTHONPATH=.
```

若 Typst 找不到，請確認 `typst` 已安裝，或使用專案內 `bin/typst`：

```
PATH="$PWD/bin:$PATH" python3 scripts/build.py
```

若 PDF build 失敗，請先查看 `dist/book_interior.typ` 附近錯誤行，再回到對應 `src/chXX.md` 修正 Markdown 或特殊符號。

若 release zip 缺少檔案，請先重新執行測試與 build，再執行：

```
python3 scripts/package_release.py
```

A.5 建議學習路線

不同讀者可以用不同順序閱讀：

| 讀者類型 | 建議順序 | 重點 | |—|—|—| | 初學者 | 第 1 章 → 第 2 章 → 第 3 章 → 第 6 章 → 第 7 章 → 第 8 章 | 先建立多 Agent、監管、微修與測試觀念，再看整合與發布。 | | 專案管理者 | 第 1 章 → 第 4 章 → 第 5 章 → 第 7 章 → 第 8 章 | 聚焦任務合約、品質管理、流程與發布。 | | 工程實作者 | 第 3 章 → 第 4 章 → 第 6 章 → 第 8 章 | 聚焦微修、Plan / Build、沙盒測試與打包。 | | 技術主編 | 第 5 章 → `docs/style_guide.md` → `qa/reports/` → 第 7 章 | 聚焦術語、審稿、記憶規則與品質閘門。 |

A.6 下一步延伸

v1.0 完成後，可以考慮：

1. 增加 EPUB 輸出。
2. 加入 GitHub Actions。
3. 將 QA 報告轉成 JSON metadata。
4. 為 Herdr 與 OpenCode 補真實操作截圖。
5. 將每章擴充到正式技術書篇幅。